

3.1 引言

在上一章中，我们展示了如何分析和设计组合逻辑。组合逻辑的输出仅取决于当前的输入值。给定以真值表或布尔方程形式给出的规格说明，我们可以创建一个优化的电路来满足该规格。

在本章中，我们将分析和设计`sequential`逻辑。时序逻辑的输出取决于当前和先前的输入值。因此，时序逻辑具有记忆功能。时序逻辑可能会显式地记住某些先前的输入，或者将先前的输入提炼为更少量的信息，称为系统的`state`。数字时序电路的状态是一组称为`state variables`的比特，其中包含解释电路未来行为所需的全部过去信息。

本章首先研究锁存器和触发器，它们是存储一位状态的简单时序电路。通常，时序电路的分析较为复杂。为简化设计，我们约束自己仅构建由组合逻辑和包含电路状态的触发器组构成的同步时序电路。本章描述了有限状态机，这是设计时序电路的一种简便方法。最后，我们分析了时序电路的速度，并讨论了并行性作为提升速度的一种方式。

3.2 锁存器与触发器

存储器的基础构建模块是`bistable`元件，这是一种具有两个稳定状态的元件。图3.1(a)展示了一个简单的双稳态元件，由一对首尾相连的反相器组成。图3.1(b)以强调对称性的方式重新绘制了同一电路。这两个反相器是`cross-coupled`连接的，即I1的输入是I2的输出，反之亦然。该电路没有输入，但有两个输出。

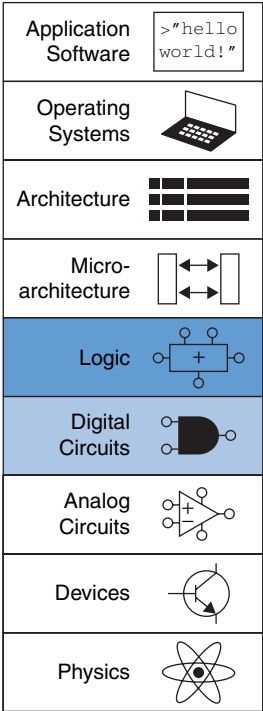
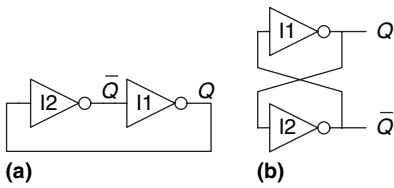


图3.1 交叉耦合反相器对



正如Y常用于组合逻辑的输出，Q常用于时序逻辑的输出。

Q 以及 $\bar{Q}$ 。分析该电路与分析组合电路不同，因为它是有环路的： Q 依赖于 Q ，而 Q 依赖于 Q 。

考虑两种情况， Q 为0或 Q 为1。通过分析每种情况的结果，我们得到：

Case I: $Q = 0$ 如图3.2(a)所示，I2接收一个FALSE输入 Q ，因此在 Q 上产生TRUE输出。I1接收一个TRUE输入 Q ，因此在 Q 上产生FALSE输出。这与原始假设 $Q = 0$ 一致，因此该情况被称为*stable*。

Case II: $Q = 1$

如图3.2(b)所示，I2接收TRUE输入并在 Q 上产生FALSE输出。I1接收FALSE输入并在 Q 上产生TRUE输出。这再次保持稳定。

由于交叉耦合反相器具有两个稳定状态，即 $Q = 0$ 和 $Q = 1$ ，因此该电路被称为双稳态。一个微妙之处在于，该电路还存在第三种可能状态，此时两个输出端均大致处于0和1的中间位置。这被称为*metastable*状态，将在第3.5.4节中讨论。

一个具有 N 个稳定状态的元件可以传递 $\log_2 N$ 比特的信息，因此一个双稳态元件存储一个比特。交叉耦合反相器的状态包含在一个二进制状态变量 Q 中。 Q 的值告诉我们解释电路未来行为所需的全部过去信息。具体来说，如果 $Q = 0$ ，它将永远保持为0；如果 $Q = 1$ ，它将永远保持为1。该电路确实还有另一个节点 Q ，但 Q 不包含任何额外信息，因为如果 Q 是

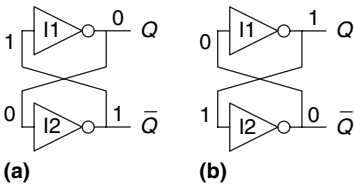


图3.2 交叉耦合反相器的双稳态操作

已知, $\bar{Q}$ 也是已知的。另一方面, Q 也是状态变量的一个可接受选择。

当电源首次施加到时序电路时, 初始状态是未知的, 通常不可预测。每次电路开启时, 该状态可能不同。

尽管交叉耦合反相器可以存储一位信息, 但由于用户没有控制状态的输入, 它们并不实用。然而, 其他双稳态元件, 如 *latches* 和 *flip-flops*, 提供了控制状态变量值的输入。本节剩余部分将讨论这些电路。

3.2.1 SR锁存器

最简单的时序电路之一是 *SR latch*, 它由两个交叉耦合的或非门组成, 如图3.3所示。该锁存器有两个输入 S 和 R , 以及两个输出 Q 和 $\bar{Q}$ 。SR锁存器与交叉耦合的反相器类似, 但其状态可通过 S 和 R 输入控制, 这两个输入 *set* 和 *reset* 输出 Q 。

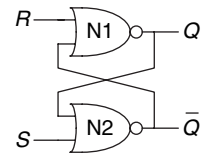


图3.3 SR锁存器原理图

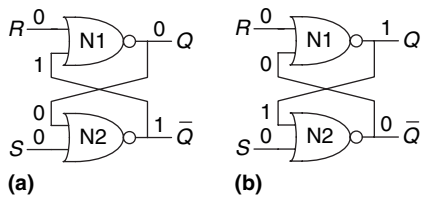
理解陌生电路的一个好方法是推导出其真值表, 因此我们从此处开始。回顾一下, 当任一输入为真时, NOR门输出为假。考虑 R 和 S 的四种可能组合。

Case I: $R = 1, S = 0$ N1看到至少一个TRUE输入, R , 因此在 Q 上产生FALSE输出。N2看到 Q 和 S 均为FALSE, 因此在 Q 上产生TRUE输出。

► *Case II:* $R = 0, S = 1$ N1接收输入0和 Q 。由于我们尚不知道 Q , 无法确定输出 Q 。N2接收到至少一个TRUE输入, S , 因此在 Q 上产生FALSE输出。现在我们可以重新审视N1, 已知两个输入均为FALSE, 因此输出 Q 为TRUE。► *Case III:* $R = 1, S = 1$ N1和N2都看到至少一个TRUE输入 (R 或 S), 因此各自产生FALSE输出。因此, $\bar{Q}$ 和 Q 均为FALSE。

► *Case IV:* $R = 0, S = 0$ N1接收输入0和 Q 。由于我们尚不知道 Q , 无法确定输出。N2接收输入0和 Q 。由于我们尚不知道 Q , 无法确定输出。现在我们

图3.4 SR锁存器的双稳态



卡住了。这让人联想到交叉耦合的反相器。但我们知道 Q 必须为0或1，因此我们可以通过检查每种子情况下的结果来解决问题。

Case IVa: $Q = 0$
由于 S 和 Q 为FALSE，N2在 Q 上产生TRUE输出，如图3.4(a)所示。现在N1接收到一个TRUE输入 $\bar{Q}$ ，因此其输出 Q 为FALSE，正如我们之前假设的那样。

Case IVb: $Q = 1$
由于 Q 为真，N2在 Q 上产生假输出，如图3.4(b)所示。现在 $\bar{N}1$ 接收到两个假输入，即 R 和 Q ，因此其输出 Q 为真，正如我们之前所假设的那样。

综合来看，假设在进入情况IV之前， Q 具有某个已知的先验值，我们称之为 Q_{prev} 。 Q_{prev} 为0或1，表示系统的状态。当 R 和 S 均为0时， Q 将记住这个旧值 Q_{prev} ，而 Q 为其补值 $\bar{Q}_{prev}$ —该电路具有记忆功能。

Case	S	R	Q	$\bar{Q}$
IV	0	0	Q_{prev}	$\bar{Q}_{prev}$
I	0	1	0	1
II	1	0	1	0
III	1	1	0	0

图3.5 SR锁存器真值表

图3.5中的真值表总结了这四种情况。输入 S 和 R 代表Set和Reset。将某位set意味着将其设为TRUE。将某位reset意味着将其设为FALSE。输出 Q 和 $\bar{Q}$ 通常是互补的。当 R 被断言时， Q 被重置为0，而 $\bar{Q}$ 则相反。当 S 被断言时， Q 被设置为1，而 $\bar{Q}$ 则相反。当两个输入都未被断言时， Q 保持其旧值 Q_{prev} 。同时断言 S 和 R 没有太大意义，因为这意味着锁存器应同时被设置和重置，这是不可能的。这个可怜的混乱电路会通过使两个输出都为0来响应。

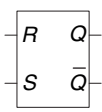


图3.6 SR锁存器符号

SR锁存器由图3.6中的符号表示。使用该符号是抽象化和模块化的一种应用。构建SR锁存器有多种方式，例如使用不同的逻辑门或晶体管。然而，任何具有图3.5真值表所指定关系以及图3.6所示符号的电路元件，都被称为SR锁存器。

与交叉耦合反相器类似，SR锁存器是一种双稳态元件，其状态的一位存储在 Q 中。然而，该状态可通过 S 和 R 输入进行控制。当 R 被断言时，状态复位为0。当 S 被断言时，状态置位为1。当两者均未被断言时，

状态保持其旧值。请注意，输入的整个历史都可以由单一状态变量 Q 来体现。无论过去发生了何种置位和复位模式，要预测SR锁存器的未来行为，只需知道它最近是被置位还是复位即可。

3.2.2 D锁存器

SR锁存器存在缺陷，因为当S和R同时被置位时，其行为异常。此外，S和R输入混淆了*what*与*when*的问题。置位任一输入不仅决定了*what*状态应为何值，还决定了*when*状态何时应改变。当这些“是什么”与“何时”的问题被分离时，电路设计会变得更加容易。图3.7(a)中的D锁存器解决了这些问题。它有两个输入：*data*输入D控制下一个状态应为何值，*clock*输入CLK控制状态何时应改变。

再次，我们通过写出图3.7(b)所示的真值表来分析锁存器。为方便起见，我们首先考虑内部节点D、S和R。如果 $CLK = 0$ ，则无论D的值如何，S和R均为FALSE。如果 $CLK = 1$ ，则根据D的值，一个与门输出TRUE，另一个输出FALSE。给定S和R，利用图3.5确定Q和 $\bar{Q}$ 。观察到当 $CLK = 0$ 时，Q记住其旧值 Q_{prev} 。当 $CLK = 1$ 时， $Q = D$ 。在所有情况下， $\bar{Q}$ 是Q的补，这符合逻辑。D锁存器避免了R和S输入同时被断言的特殊情况。

综合来看，时钟控制着数据何时通过锁存器。当 $CLK = 1$ 时，锁存器处于*transparent*状态。D处的数据会直接传输到Q，仿佛锁存器只是一个缓冲器。当 $CLK = 0$ 时，锁存器处于*opaque*状态。它会阻止新数据流向Q，而Q则保持旧值。因此，D锁存器有时被称为*transparent latch*或*level-sensitive*锁存器。D锁存器的符号如图3.7(c)所示。

D锁存器在 $CLK = 1$ 时连续更新其状态。我们将在本章后面看到，仅在特定时刻更新状态是有用的。下一节描述的D触发器正是实现这一功能的。

有些人将锁存器称为开启或关闭，而非透明或不透明。然而，我们认为这些术语存在歧义——*open*是指像敞开的门一样透明，还是像开路一样不透明？

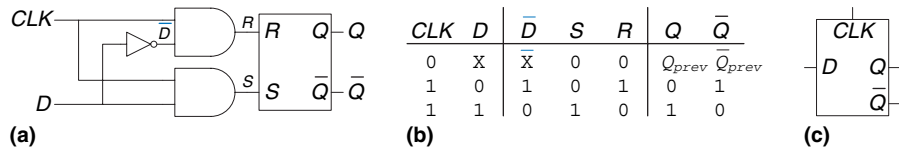


图3.7 D锁存器：(a) 原理图，(b) 真值表，(c) 符号

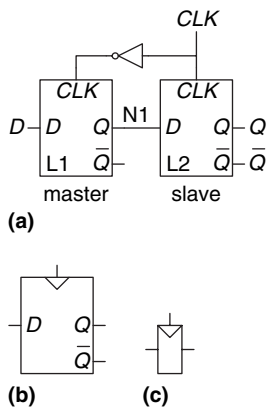


图3.8 D触发器：(a) 原理图，(b) 符号，(c) 简化符号

*flip-flops*与*latches*之间的精确区别有些模糊，并随时间演变。在行业通用用法中，触发器是*edge-triggered*。换言之，它是一种具有*clock*输入的双稳态元件。触发器的状态仅在时钟边沿（例如时钟从0上升至1时）发生变化。没有边沿触发时钟的双稳态元件通常称为锁存器。

术语*flip-flop*或*latch*本身通常分别指代*D flip-flop*或*D latch*，因为这是实践中最为常用的类型。

3.2.3 D触发器

如图3.8(a)所示，一个*D flip-flop*可以由两个背靠背的D锁存器构成，这两个锁存器由互补时钟控制。第一个锁存器L1称为*leader*。第二个锁存器L2称为*follower*，因为它跟随L1的动作。

它们之间的节点命名为N1。D触发器的符号如图3.8(b)所示。当不需要Q输出时，该符号通常简化为图3.8(c)的形式。

当CLK = 为0时，主锁存器（L1）处于透明状态，从锁存器（L2）处于不透明状态。因此，D处的任何值都会传播到N1。当CLK = 为1时，主锁存器（L1）变为不透明，从锁存器（L2）变为透明。N1处的值传播到Q，但N1与D断开连接。因此，在时钟从0上升为1之前瞬间位于D的值，会在时钟上升后立即复制到Q。在其他所有时间，Q保持其原有值，因为总有一个不透明的锁存器阻断D与Q之间的路径。

换句话说，*a D flip-flop copies D to Q on the rising edge of the clock and remembers its state at all other times*。请反复阅读这一定义直至牢记；初学数字设计者最常见的问题之一就是忘记触发器的作用。为简洁起见，时钟的上升沿常简称为*clock edge*。D输入指定新状态为何值。时钟边沿指示何时应更新状态。

D触发器也称为*edge-triggered flip-flop*或*positive edge-triggered flip-flop*。符号中的三角形表示边沿触发的时钟输入。当不需要时，Q输出通常被省略。

示例3.1 触发器晶体管数量

构建本节所述的D触发器需要多少个晶体管？

解决方案 一个与非门或非门使用四个晶体管。一个非门使用两个晶体管。一个与门由一个与非门和一个非门构成，因此使用六个晶体管。SR锁存器使用两个或非门，即八个晶体管。D锁存器使用一个SR锁存器、两个与门和一个非门，共22个晶体管。D触发器使用两个D锁存器和一个非门，共46个晶体管。第3.2.7节描述了一种更高效的CMOS实现方式，使用传输门。

3.2.4 寄存器

一个N位寄存器是由N个触发器组成的存储库，这些触发器共享一个公共的CLK输入，使得寄存器的所有位同时更新。寄存器是大多数时序电路的关键构建模块。

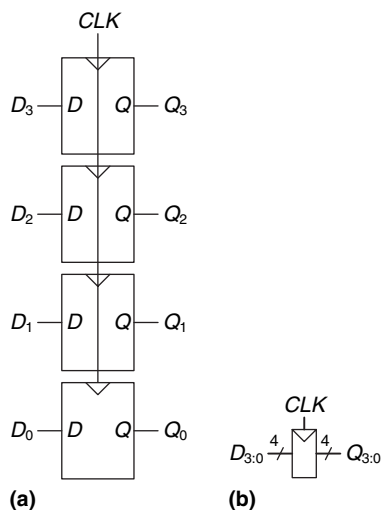


图3.9 一个4位寄存器：(a) 原理图和(b) 符号

图3.9展示了一个四位寄存器的原理图和符号，其输入为 $D_{3:0}$ ，输出为 $Q_{3:0}$ 。 $D_{3:0}$ 和 $Q_{3:0}$ 均为4位总线。

3.2.5 使能触发器

一个*enabled flip-flop*增加了一个名为 EN 或 $ENABLE$ 的输入，用于确定数据是否在时钟边沿加载。当 EN 为真时，使能触发器像普通D触发器一样工作。当 EN 为假时，使能触发器忽略时钟并保持其状态。使能触发器在我们希望仅在某些时候（而非每个时钟边沿）向触发器加载新值时非常有用。

图3.10展示了两种通过D触发器和额外门电路构建使能触发器的方法。在图3.10(a)中，输入多路选择器在 EN 为真时传递值 D ，或在 EN 为假时从 Q 循环旧状态。在图3.10(b)中，时钟为 $gated$ 。如果 EN 为真，则触发器的 CLK 输入正常翻转。如果 EN 为假，则 CLK 输入也为假，触发器保持其旧值。注意， EN 必须

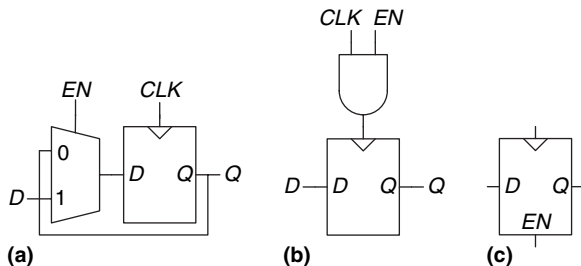


图3.10 使能触发器：
(a, b) 示意图，(c) 符号

在 $CLK = 1$ 期间不要改变，以免触发器在错误的时间看到时钟 $glitch$ （切换）。通常，对时钟进行逻辑操作是一个坏主意。时钟门控会延迟时钟并可能导致时序错误，正如我们将在第3.5.3节中看到的，因此只有在确保自己清楚在做什么时才这样做。使能触发器的符号如图3.10(c)所示。

3.2.6 可复位触发器

一个 $resettable\ flip-flop$ 增加了另一个输入，称为 $RESET$ 。当 $RESET$ 为 $FALSE$ 时，可复位触发器行为与普通D触发器相同。当 $RESET$ 为 $TRUE$ 时，可复位触发器忽略 D 并将输出复位为0。可复位触发器在系统首次启动时，若希望将所有触发器强制置为已知状态（即0）时非常有用。

此类触发器可以是 $synchronously$ 或 $asynchronously\ resettable$ 。同步可复位触发器仅在 CLK 的上升沿自行复位。异步可复位触发器则在 $RESET$ 变为真时立即自行复位，与 CLK 无关。

图3.11(a)展示了如何通过一个普通D触发器和与门构建同步复位触发器。当信号 $RESET$ 为 $FALSE$ 时，与门强制将0输入触发器。当 $RESET$ 为 $TRUE$ 时，与门将 D 传递至触发器。在此示例中， $RESET$ 是 $active\ low$ 信号，意味着复位信号在值为0而非1时执行其功能——即复位触发器。通过添加反相器，该电路可改为接受高电平有效复位信号。图3.11(b)和3.11(c)展示了具有高电平有效复位的可复位触发器符号。

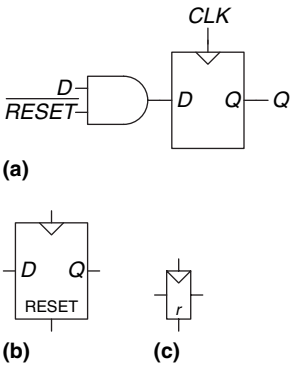


图3.11 同步可复位触发器：(a) 原理图，(b, c) 符号

异步可复位触发器需要修改触发器的内部结构，留待你在练习3.13中自行设计。同步和异步可复位触发器通常作为标准组件供设计者使用。

正如您可能想象的那样，可置位触发器也偶尔会被使用。当 SET 信号有效时，它们会将1加载到触发器中，并且同样分为同步和异步两种类型。可复位和可置位触发器也可能具有使能输入，并且可以组合成 N 位寄存器。

3.2.7 晶体管级锁存器与触发器设计*

示例3.1表明，用逻辑门构建锁存器和触发器需要大量晶体管。但锁存器的基本作用是透明或不透明，很像一个开关。回想一下

从第1.7.7节可知，传输门是构建CMOS开关的一种高效方式，因此我们可以预期利用传输门来减少晶体管数量。

一个紧凑的D锁存器可以由单个传输门构成，如图3.12(a)所示。当 $CLK = 1$ 且 $\overline{CLK} = 0$ 时，传输门导通，因此 D 流向 Q ，锁存器处于透明状态。当 $CLK = 0$ 且 $\overline{CLK} = 1$ 时，传输门关断，因此 Q 与 D 隔离，锁存器处于不透明状态。该锁存器存在两个主要限制：

Floating output node: 当锁存器不透明时， Q 的值不会由任何门电路保持。因此， Q 被称为 *floating* 或 *dynamic* 节点。一段时间后，噪声和电荷泄漏可能会干扰 Q 的值。► **No buffers:** 缺少缓冲器已导致多款商用芯片出现故障。一个将 D 拉至负电压的噪声尖峰可能导通 nMOS 晶体管，使锁存器变为透明，即使 $CLK = 0$ 时也是如此。同样， D 上高于 V_{DD} 的尖峰可能导通 pMOS 晶体管，即使 $CLK = 0$ 时也是如此。而且传输门是对称的，因此 Q 上的噪声可能反向驱动它，影响输入 D 。一般规则是：传输门的输入或时序电路的状态节点绝不应暴露于外部世界，因为外部噪声很常见。

图3.12(b)展示了一种更稳健的12晶体管D锁存器，用于现代商业芯片。它仍然以时钟传输门为核心，但增加了反相器I1和I2来缓冲输入和输出。锁存器的状态保存在节点N1上。反相器I3和三态缓冲器T1提供反馈，使N1成为一个 *static node*。如果在 $CLK = 0$ 时N1上出现少量噪声，T1会将N1驱动回有效的逻辑值。

图3.13展示了一个由 CLK 和 $\overline{CLK}$ 控制的两个静态锁存器构成的D触发器。一些冗余的内部反相器已被移除，因此该触发器仅需20个晶体管。

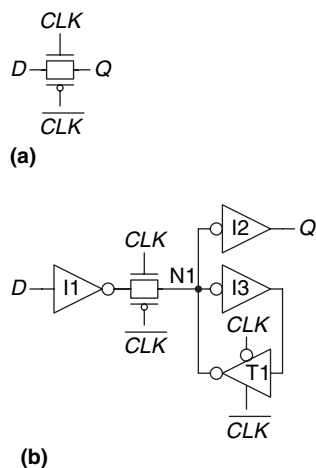


图3.12 D锁存器原理图

该电路假设 CLK 和 $\overline{CLK}$ 均可用。若不可用，则需要额外两个晶体管来反相时钟信号。

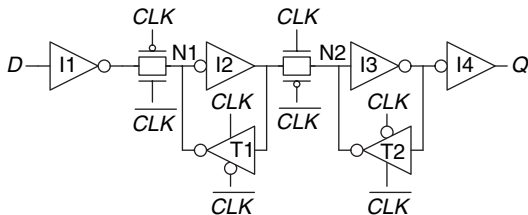


图3.13 D触发器原理图

3.2.8 综合应用

锁存器和触发器是时序电路的基本构建模块。请记住，D锁存器是电平敏感的，而D触发器是边沿触发的。当 $CLK = 1$ 时，D锁存器是透明的，允许输入 D 直接传输到输出 Q 。D触发器在 CLK 的上升沿将 D 复制到 Q 。在其他所有时间，锁存器和触发器保持其原有状态。寄存器是由多个共享同一 CLK 信号的D触发器组成的阵列。

示例3.2 触发器与锁存器比较

Ben Bitdiddle 将图 3.14 所示的 D 和 CLK 输入应用于一个 D 锁存器和一个 D 触发器。请帮助他确定每个设备的输出 Q 。

解答 图3.15展示了输出波形，假设 Q 对输入变化的响应存在微小延迟。箭头指示了输出变化的原因。 Q 的初始值未知，可能为0或1，如一对水平线所示。首先考虑锁存器。在 CLK 的第一个上升沿， $D = 0$ ，因此 Q 必然变为0。每当 D 在 $CLK = 1$ 时发生变化， Q 也随之变化。当 D 在 $CLK = 0$ 时发生变化， D 被忽略。接下来考虑触发器。在 CLK 的每个上升沿， D 被复制到 Q 。在其他所有时间， Q 保持其状态。

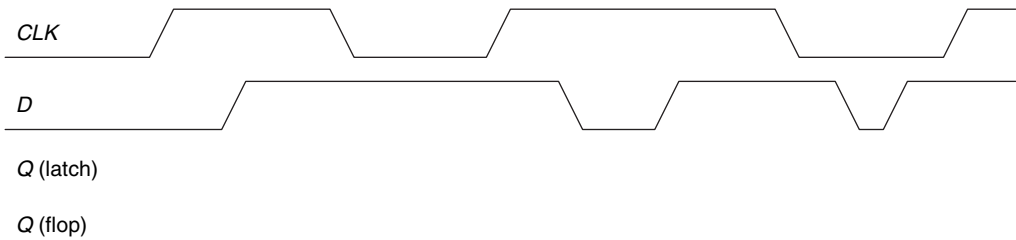


图3.14 示例波形

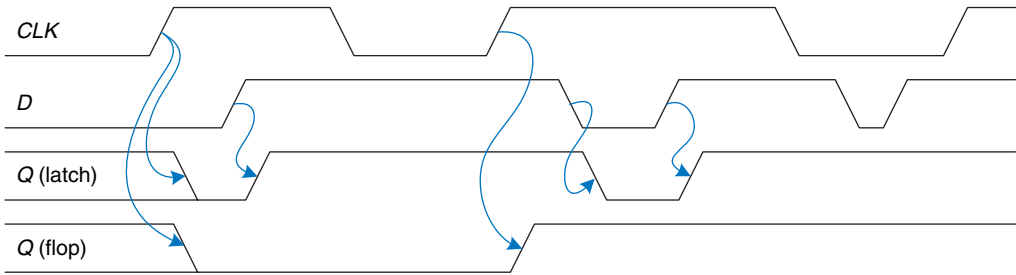


图3.15 解波形

3.3 同步逻辑设计

一般来说，时序电路包括所有非组合电路——即那些输出无法仅通过当前输入确定的电路。有些时序电路相当古怪。本节首先探讨一些这类奇特电路，然后介绍同步时序电路的概念及动态约束。通过遵循同步时序电路的约束，我们可以开发出简单、系统的方法来分析 and 设计时序系统。

3.3.1 一些有问题的电路

示例3.3 无稳态电路

Alyssa P. Hacker 遇到了三个误接的反相器，它们首尾相连形成了一个环路，如图 3.16 所示。第三个反相器的输出 *fed back* 连接到第一个反相器。每个反相器的传播延迟为 1 ns。请帮助 Alyssa 确定该电路的功能。

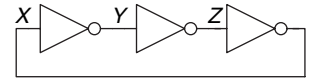


图3.16 三反相器环路

解：假设节点X初始为0。那么，Y = 为1，Z = 为0，因此X = 为1，这与我们最初的假设不一致。该电路没有稳定状态，被称为*unstable*或*astable*。图3.17展示了该电路的行为。如果X在时间0上升，Y将在1纳秒时下降，Z将在2纳秒时上升，X将在3纳秒时再次下降。接着，Y将在4纳秒时上升，Z将在5纳秒时下降，X将在6纳秒时再次上升；然后，这一模式将重复。每个节点在0和1之间振荡，其 *period*（重复时间）为6纳秒。该电路被称为*ring oscillator*。

环形振荡器的周期取决于每个反相器的传播延迟。该延迟受反相器的制造工艺、电源电压甚至温度的影响。因此，环形振荡器的周期难以精确预测。简而言之，环形振荡器是一种零输入、单输出且输出周期性变化的时序电路。

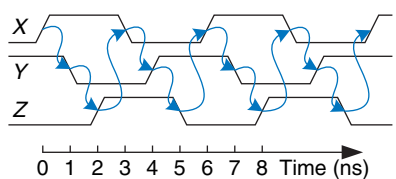
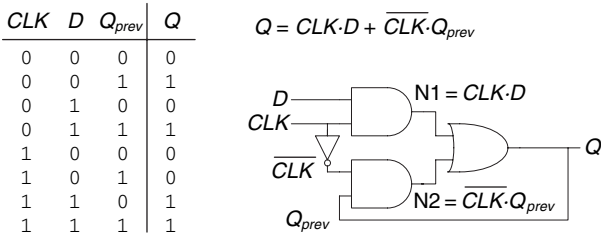


图3.17 环形振荡器波形

示例3.4 竞争条件

Ben Bitdiddle 设计了一个新的 D 锁存器，他声称该锁存器比图 3.7 中的更好，因为它使用了更少的门。他编写了真值表，以根据两个输入 *D* 和 *CLK* 以及旧状态来找出输出 *Q*。

图3.18 一个改进的 (?)
D锁存器



锁存器， Q_{prev} 。基于该真值表，他推导出了布尔方程。他通过反馈输出Q得到 Q_{prev} 。他的设计如图3.18所示。该锁存器能否正确工作，且不受每个门延迟的影响？

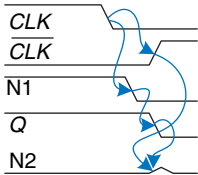


图3.19 锁存器波形图 说明竞争条件

解图3.19显示该电路存在一个*race condition*，当某些门电路比其他门电路更慢时会导致其失效。假设 $CLK = D = 1$ 。锁存器处于透明状态，将D传递过去使得 $Q = 1$ 。此时，CLK下降。锁存器应保持其旧值，使 $Q = 1$ 。然而，假设从CLK到CLK的反相器延迟相比于与门和或门的延迟较长。那么节点N1和Q可能在CLK上升之前都下降。在这种情况下，N2将永远不会上升，Q被卡在0。

这是一个*asynchronous*电路设计的示例，其中输出直接反馈到输入。异步电路因存在竞争条件而臭名昭著，在这种条件下，电路的行为取决于通过逻辑门的两条路径中哪一条最快。一个电路可能正常工作，而另一个看似相同但由延迟略有不同的门构成的电路却可能无法工作。或者，该电路可能仅在特定温度或电压下（此时延迟恰好合适）才能工作。这些故障极难排查。

3.3.2 同步时序电路

前两个示例包含称为*cyclic paths*的循环，其中输出直接反馈回输入。它们是时序电路而非组合电路。组合逻辑没有循环路径和竞争。若将输入施加于组合逻辑，输出总会在传播延迟内稳定至正确值。然而，具有循环路径的时序电路可能产生不良竞争或不稳定行为。分析此类电路中的问题非常耗时，且许多聪明人曾在此犯错。

为了避免这些问题，设计者通过在路径中插入寄存器来打破循环路径。这将电路转换为

组合逻辑与寄存器的集合。寄存器包含系统的状态，该状态仅在时钟边沿发生变化，因此我们称状态与时钟同步。若时钟足够慢，使得所有寄存器的输入在下一个时钟边沿之前稳定，则所有竞争条件均被消除。采用这种始终在反馈路径中使用寄存器的规范，引出了同步时序电路的形式化定义。

回顾一下，电路由其输入和输出端子以及其功能和时序规范定义。

时序电路具有一组有限的离散状态 $states \{S_0, S_1, S_2, \dots\}$ 。

*synchronous sequential circuit*有一个时钟输入，其上升沿指示状态发生转换的时间序列。我们常用术语*current state*和*next state*来区分系统当前的状态与下一个时钟沿将进入的状态。功能规范详细说明了对于当前状态和输入值的每种可能组合，下一个状态以及每个输出的值。时序规范包括一个上限 t_{pcq} 和一个下限 t_{ccq} ，表示从时钟上升沿到*output*变化的时间，以及*setup*和*hold*时间，即 t_{setup} 和 t_{hold} ，它们指示*inputs*相对于时钟上升沿必须保持稳定的时间。

*synchronous sequential circuit composition*的规则告诉我们，如果一个电路由相互连接的电路元件组成，且满足以下条件，则该电路是一个同步时序电路：

每个电路元件要么是寄存器，要么是组合电路

至少有一个电路元件是寄存器 ▶ 所有寄存器接收相同的时钟信号 ▶ 每个循环路径至少包含一个寄存器

非同步的时序电路称为*asynchronous*。

触发器是最简单的同步时序电路。它有一个输入*D*、一个时钟*CLK*、一个输出*Q*以及两个状态{0, 1}。触发器的功能规范是：下一状态为*D*，输出*Q*为当前状态，如图3.20所示。

我们通常将当前状态变量称为*S*，将下一状态变量称为*S'*。在这种情况下，*S*后的撇号表示下一状态，而非取反。时序电路的时序分析将在3.5节中进行。

另外两种常见的同步时序电路类型被称为有限状态机和流水线。这些将在本章后面部分介绍。

t_{pcq} 表示从时钟到*Q*的传播时间，其中*Q*指示同步时序电路的输出。 t_{ccq} 表示从时钟到*Q*的污染时间。这些类似于组合逻辑中的 t_{pd} 和 t_{cd} 。

同步时序电路的这个定义虽然足够，但比必要的条件更为严格。例如，在高性能微处理器中，某些寄存器可能会接收延迟或门控时钟，以榨取最后一点性能或功耗。同样，有些微处理器使用锁存器而非寄存器。然而，这一定义足以涵盖本书中涉及的所有同步时序电路，以及大多数商用数字系统。

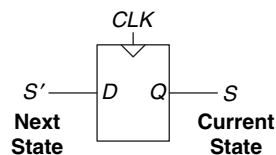


图3.20 触发器当前状态和下一状态

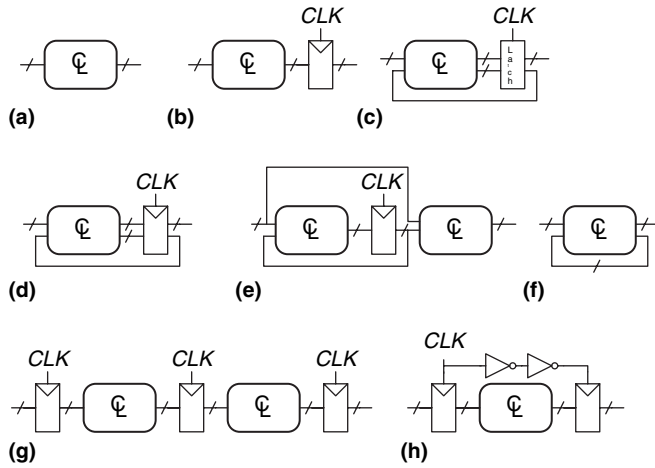


图3.21 示例电路

示例3.5 同步时序电路

图3.21中的哪些电路是同步时序电路？

解决方案 电路(a)是组合逻辑电路，而非时序电路，因为它没有寄存器。(b)是一个无反馈的简单时序电路。(c)既不是组合逻辑电路，也不是同步时序电路，因为它包含一个既非寄存器也非组合逻辑的锁存器。(d)和(e)是同步时序逻辑电路；它们是有限状态机的两种形式，将在第3.4节中讨论。(f)既不是组合逻辑电路，也不是同步时序电路，因为它有一条从组合逻辑输出回到同一逻辑输入的循环路径，但路径中没有寄存器。(g)是流水线形式的同步时序逻辑电路，我们将在第3.6节中学习。(h)严格来说不是同步时序电路，因为第二个寄存器接收的时钟信号与第一个不同，延迟了两个反相器的延迟时间。

3.3.3 同步与异步电路

异步设计在理论上比同步设计更具通用性，因为系统时序不受时钟寄存器的限制。正如模拟电路比数字电路更具通用性（模拟电路可使用任意电压），异步电路比同步电路更具通用性（异步电路可采用任意形式的反馈）。然而，实践证明同步电路比异步电路更易于设计和应用，正如数字电路比模拟电路更易实现。尽管对异步电路的研究已持续数十年，但几乎所有数字系统本质上仍是同步的。

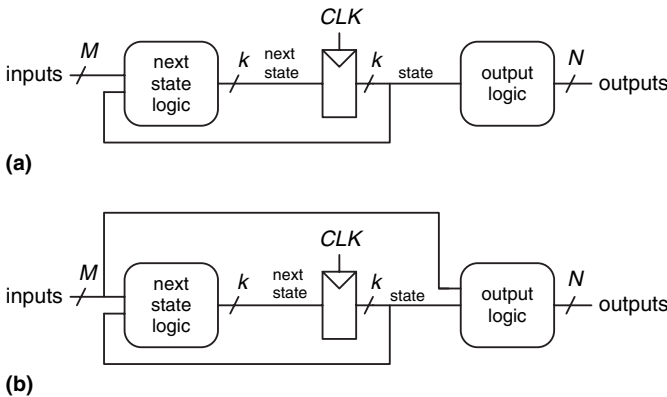
当然，当在不同时钟的系统之间通信或在任意时间接收输入时，异步电路偶尔是必要的，就像在与连续电压的现实世界通信时模拟电路是必要的一样。此外，异步电路的研究持续产生有趣的见解，其中一些也能改进同步电路。

3.4 有限状态机

同步时序电路可以绘制成图3.22所示的形式。这些形式被称为 *finite state machines* (FSMs)。之所以得名，是因为具有 k 个寄存器的 ^{theory} 电路可以处于有限数量 (2^k) 的唯一状态之一。FSM 有 M 个输入、 N 个输出和 k 位状态。它还会接收时钟信号，并可选地接收复位信号。FSM 由两个组合逻辑模块 *next state logic* 和 *output logic* 以及一个存储状态的寄存器组成。在每个时钟边沿，FSM 会进入下一个状态，该状态是根据当前状态和输入计算得出的。有限状态机根据其功能规范可分为两大类。在 *Moore machines* 中，输出仅取决于机器的当前状态。在 *Mealy machines* 中，输出同时取决于当前状态和当前输入。有限状态机提供了一种系统化的方法，用于根据功能规范设计同步时序电路。本节剩余部分将从示例开始解释该方法。

3.4.1 FSM 设计示例

为了说明有限状态机 (FSM) 的设计，考虑为校园繁忙路口设计交通灯控制器的的问题。工程专业的学生们正在宿舍与学术楼实验室之间悠闲地穿行。



摩尔和米利机以其推广者命名，这些研究者曾在贝尔实验室开发了 *automata*，即状态机的数学基础。

爱德华·F·摩尔 (1925–2003)，不要与英特尔创始人戈登·摩尔混淆，于1956年发表了他的开创性文章《关于序列机的思想实验》。随后，他成为威斯康星大学数学与计算机科学教授。

乔治·H·米利 (1927–2010) 于1955年发表了他的文章《一种综合时序电路的方法》。随后，他为IBM 704计算机编写了第一个贝尔实验室操作系统。之后，他加入了哈佛大学。

图3.22 有限状态机：(a) Moore机，
(b) 米利机

大道上，他们正忙着阅读最喜欢的教科书中的FSM内容，没有注意看路。足球运动员们在布拉瓦多大道上的运动场和食堂之间穿梭，他们来回抛球，同样没有注意看路。这两条路的交叉口已经发生了多起严重受伤事件，学生事务主任要求本·比特迪德尔在出现死亡事故之前安装一个交通信号灯。

本决定用有限状态机解决该问题。他在学术大道和布拉瓦多大街分别安装了交通传感器 T_A 和 T_B 。每个传感器在有学生时输出TRUE，街道空置时输出FALSE。他还安装了两个交通灯 L_A 和 L_B 来控制交通。每个灯接收数字输入，指定其应显示绿灯、黄灯或红灯。因此，他的有限状态机有两个输入 T_A 和 T_B ，以及两个输出 L_A 和 L_B 。带有传感器和灯的路口如图3.23所示。本提供了一个周期为5秒的时钟。在每个时钟节拍（上升沿），灯可能根据交通传感器改变状态。他还提供了一个复位按钮，以便物理设施技术人员在启动时将控制器置于已知初始状态。图3.24展示了该状态机的黑盒视图。

Ben的下一步是绘制*state transition diagram*，如图3.25所示，以标示系统的所有可能状态及这些状态之间的转换。当系统重置时，Academic Ave.上的灯为绿色，Bravado Blvd.上的灯为红色。每隔5秒，控制器会检查交通模式并决定下一步操作。只要

图3.23 校园地图

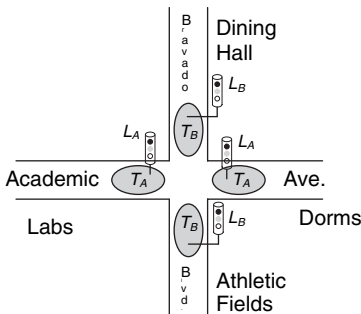
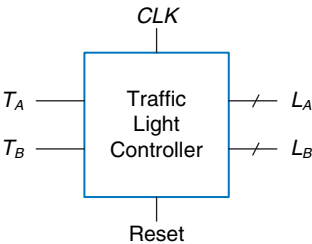


图3.24 有限状态机的黑盒视图



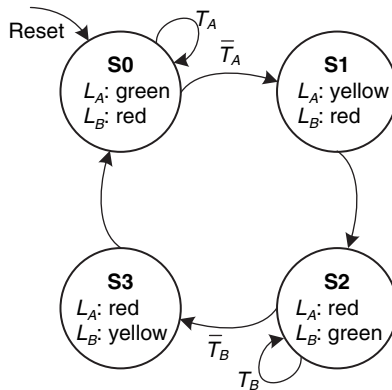


图3.25 状态转移图

学术大道上有车辆时，交通灯不会变化。当学术大道上不再有车辆时，学术大道的黄灯亮起5秒，随后转为红灯，而布拉瓦多大道的绿灯亮起。同样，只要布拉瓦多大道上有车辆，其绿灯就会保持，随后转为黄灯，最终变为红灯。

在状态转换图中，圆圈表示状态，弧线表示状态之间的转换。转换发生在时钟上升沿；我们无需在图中标出时钟，因为在同步时序电路中时钟始终存在。此外，时钟仅控制转换发生的时机，而图则指示哪些转换会发生。标有Reset的弧线从外部指向状态S0，表示无论系统之前处于何种状态，复位后都应进入该状态。如果某个状态有多条离开的弧线，则弧线上会标注触发每次转换的输入条件。例如，在状态S0时，若 T_A 为真则系统保持该状态，若 $\bar{T}_A$ 为假则转移到S1。如果某个状态只有一条离开的弧线，则该转换始终发生，与输入无关。例如，在状态S1时，系统总会在时钟边沿转移到S2。处于特定状态时输出的值会在状态中标注。例如，在状态S2时， L_A 为红色， L_B 为绿色。

Ben将状态转换图重写为state transition table（表3.1），该表针对每个状态和输入指示了下一个状态S'应为何值。注意，当下一个状态不依赖于特定输入时，表中使用了无关符号（X）。此外，请注意Reset已从表中省略。相反，我们使用可复位的触发器，这些触发器在复位时始终进入状态S0，与输入无关。

状态转换图是抽象的，它使用了标记为{S0, S1, S2, S3}的状态和标记为{red, yellow, green}的输出。要构建实际电路，必须为状态和输出分配binary encodings。Ben选择了表3.2和表3.3中给出的简单编码。每个状态和输出用两位编码： $S_{1:0}$ 、 $L_{A1:0}$ 和 $L_{B1:0}$

当前状态通常简称为系统的state。

Notice that states are designated as S0, S1, etc. The subscripted versions, S_0 , S_1 , etc., refer to the state bits.

表3.1 状态转换表

Current State S	Inputs		Next State S'
	T_A	T_B	
S0	0	X	S1
S0	1	X	S0
S1	X	X	S2
S2	X	0	S3
S2	X	1	S2
S3	X	X	S0

表3.2 状态编码

State	Encoding $S_{1:0}$
S0	00
S1	01
S2	10
S3	11

表3.3 输出编码

Output	Encoding $L_{1:0}$
green	00
yellow	01
red	10

Ben更新状态转换表以使用这些二进制编码，如表3.4所示。修订后的状态转换表是一个指定下一状态逻辑的真值表。它将下一状态 S' 定义为当前状态 S 和输入的函数。

从该表可以直接读出以积和形式表示的下一状态布尔方程。

$$\begin{aligned} S'_1 &= \bar{S}_1 S_0 + S_1 \bar{S}_0 \bar{T}_B + S_1 \bar{S}_0 T_B \\ S'_0 &= \bar{S}_1 \bar{S}_0 \bar{T}_A + S_1 \bar{S}_0 \bar{T}_B \end{aligned} \tag{3.1}$$

可以使用卡诺图简化方程。然而，通过观察通常更容易。例如, S'_1 方程中的 T_B 和 $\bar{T}_B$ 项显然是冗余的。因此, S'_1 简化为一个异或运算。方程 3.2 给出了简化后的 *next state equations*。

表3.4 带二进制编码的状态转换表

Current State		Inputs		Next State	
S_1	S_0	T_A	T_B	S'_1	S'_0
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

表3.5 输出表

Current State		Outputs			
S_1	S_0	L_{A1}	L_{A0}	L_{B1}	L_{B0}
0	0	0	0	1	0
0	1	0	1	1	0
1	0	1	0	0	0
1	1	1	0	0	1

$$\begin{aligned} S'_1 &= S_1 \oplus S_0 \\ S'_0 &= \bar{S}_1 \bar{S}_0 \bar{T}_A + S_1 \bar{S}_0 \bar{T}_B \end{aligned} \quad (3.2)$$

同样，Ben编写了一个*output table*（表3.5），该表指示了在每个状态下输出应该是什么。同样，直接读取并简化输出的布尔方程也很简单。例如，观察到 L_{A1} 仅在 S_1 为TRUE的行上为TRUE。

$$\begin{aligned} L_{A1} &= S_1 \\ L_{A0} &= \bar{S}_1 S_0 \\ L_{B1} &= \bar{S}_1 \\ L_{B0} &= S_1 S_0 \end{aligned} \quad (3.3)$$

最后，Ben以图3.22(a)的形式勾勒出他的摩尔有限状态机。首先，他绘制了2位状态寄存器，如图3.26(a)所示。在每个时钟边沿，状态寄存器将下一状态 $S'_{1:0}$ 复制为状态 $S_{1:0}$ 。状态寄存器接收同步或异步复位，以便在启动时初始化有限状态机。然后，他根据方程3.2绘制下一状态逻辑，该逻辑从当前状态和输入计算出下一状态，如图3.26(b)所示。最后，他根据方程3.3绘制输出逻辑，该逻辑从当前状态计算出输出，如图3.26(c)所示。

图3.27展示了一个时序图，说明了交通灯控制器经历一系列状态的过程。该图显示了CLK、Reset、输入 T_A 和 T_B 、下一状态 S' 、状态 S 以及输出 L_A 和 L_B 。箭头表示因果关系；例如，状态变化导致输出变化，输入变化导致下一状态变化。虚线表示状态变化时CLK的上升沿。

时钟周期为5秒，因此交通灯每5秒最多变化一次。当有限状态机首次启动时，其状态未知，如问号所示。因此，系统应重置以使其进入已知状态。在此时序图中， S

此示意图使用了若干与门，其输入端带有小圆圈。这些与门可能由与门和输入反相器构成，也可能由或非门和用于非圆圈输入端的反相器构成，或者采用其他门电路的组合。最佳选择取决于具体的实现技术。

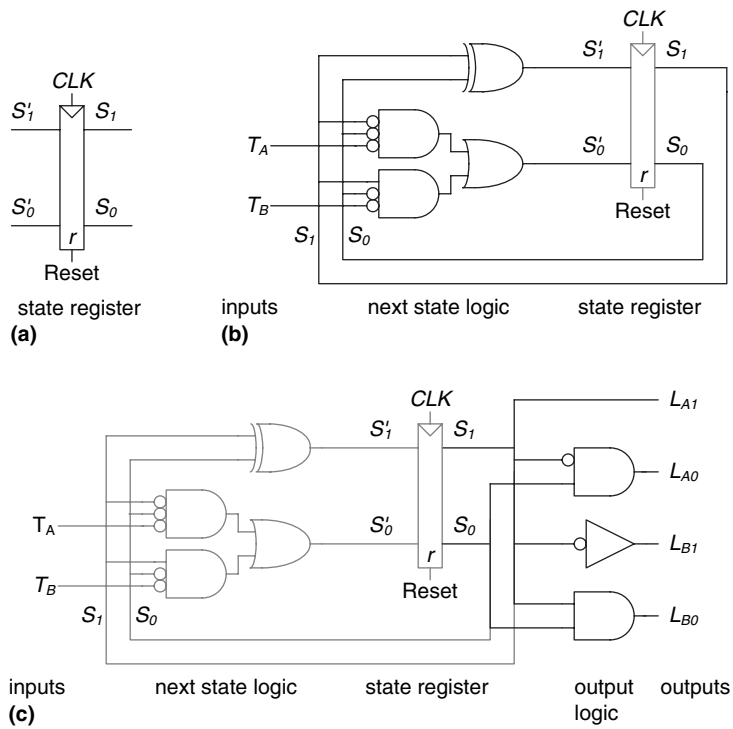


图3.26 交通灯控制器的状态机电路

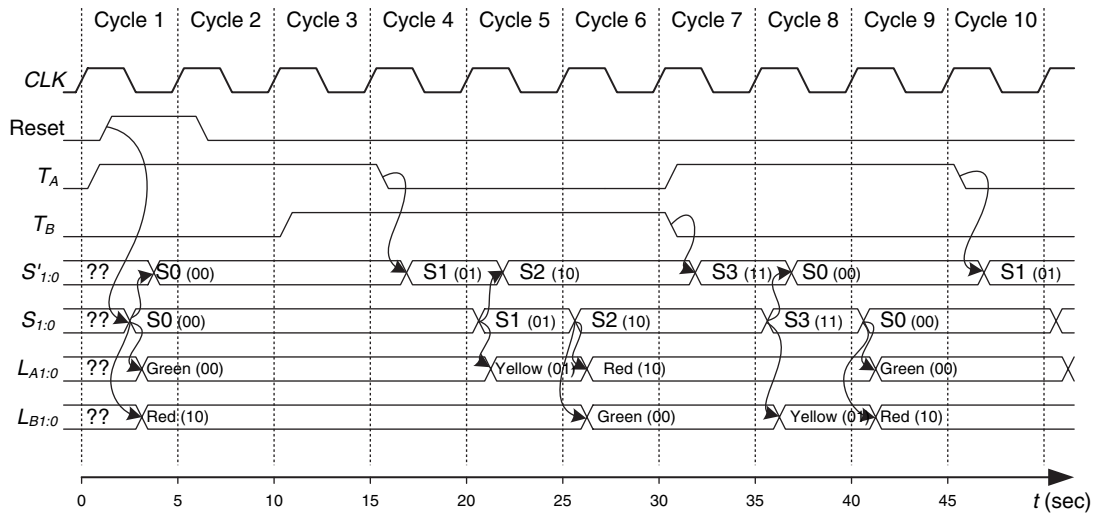


图3.27 交通灯控制器时序图

立即重置为S0，表明正在使用异步可复位触发器。在状态S0中，灯 L_A 为绿色，灯 L_B 为红色。

在此示例中，交通立即到达Academic Ave.。因此，控制器保持在状态S0，即使有车辆到达Bravado Blvd.并开始等待， L_A 仍保持绿灯。15秒后，Academic Ave.上的车辆全部通过， T_A 变为低电平。在下一个时钟边沿，控制器进入状态S1，将 L_A 变为黄灯。再过5秒，控制器进入状态S2，此时 L_A 变为红灯， L_B 变为绿灯。控制器在状态S2等待，直到Bravado Blvd.上的所有车辆通过。随后进入状态S3，将 L_B 变为黄灯。5秒后，控制器进入状态S0，将 L_B 变为红灯， L_A 变为绿灯。此过程重复进行。

尽管本竭尽全力，学生们仍不注意交通信号灯，碰撞事故持续发生。学生事务主任随后要求他和阿莉莎设计一台投石机，将工程系学生直接从宿舍屋顶抛入实验室的窗户，完全绕过那个麻烦的十字路口。不过，那是另一本教科书的内容了。

3.4.2 状态编码

在前一个例子中，状态和输出编码是任意选择的。不同的选择会导致不同的电路。一个自然的问题是，如何确定能产生逻辑门最少或传播延迟最短的编码。遗憾的是，除了尝试所有可能性之外，没有简单的方法来找到最佳编码，而当状态数量很大时，这并不可行。然而，通常可以通过观察来选择一个良好的编码，使相关的状态或输出共享比特位。计算机辅助设计（CAD）工具也擅长搜索可能的编码集并选择一个合理的编码。

状态编码中的一个重要决策是选择二进制编码还是独热编码。在交通灯控制器示例中使用的**binary encoding**，每个状态被表示为一个二进制数。由于 K 个二进制数可以用 $\log_2 K$ 位表示，因此具有 K 个状态的系统只需要 $\log_2 K$ 位状态。

在**one-hot encoding**中，每个状态使用一个独立的位来表示。它被称为**one-hot**，因为任何时候只有一位是“热”的或为真。例如，一个具有三个状态的一热编码FSM的状态编码为001、010和100。每个状态位存储在一个触发器中，因此一热编码比二进制编码需要更多的触发器。然而，采用一热编码时，下一状态和输出逻辑通常更简单，因此所需的门电路更少。最佳的编码选择取决于具体的FSM。



示例3.6 FSM状态编码

一个**divide-by-N counter**有一个输出且没有输入。输出Y在每 N 个时钟周期中有一个周期为高电平。换句话说，该输出将时钟频率除以 N 。图3.28展示了一个三分频计数器的波形和状态转换图。请使用二进制编码和独热编码分别绘制该计数器的电路设计草图。

图3.28 三分频计数器 (a)波形图和(b)状态转换图

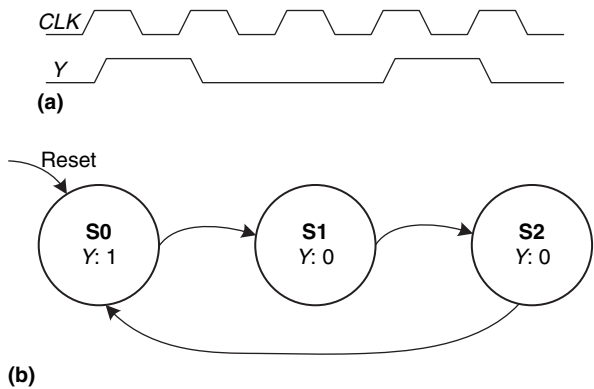


表3.6 三分频计数器 状态转换表

Current State	Next State
S0	S1
S1	S2
S2	S0

表3.7 三分频计数器输出表

Current State	Output
S0	1
S1	0
S2	0

解决方案 表3.6和表3.7分别展示了编码前的抽象状态转换表和输出表。

表3.8比较了三种情况的二进制编码和独热编码状态。

二进制编码使用两位状态。采用此编码，状态转换表如表3.9所示。注意没有输入；下一状态仅取决于当前状态。输出表留给读者作为练习。下一状态和输出方程如下：

$$\begin{aligned} S'_1 &= \bar{S}_1 S_0 \\ S'_0 &= \bar{S}_1 \bar{S}_0 \end{aligned} \tag{3.4}$$

$$Y = \bar{S}_1 \bar{S}_0 \tag{3.5}$$

独热编码使用三位状态。该编码的状态转换表如表3.10所示，输出表同样留给读者作为练习。次态方程和输出方程如下：

$$\begin{aligned} S'_2 &= S_1 \\ S'_1 &= S_0 \\ S'_0 &= S_2 \end{aligned} \tag{3.6}$$

$$Y = S_0 \tag{3.7}$$

图3.29展示了每种设计的原理图。请注意，二进制编码设计的硬件可以优化，使Y和S'_0共享同一个门电路。同时，观察到位独热编码需要同时具备可置位(s)和可复位(r)的触发器，以便在复位时将机器初始化为S0状态。最佳实现方案取决于门电路与触发器的相对成本，但在此特定示例中，位独热编码设计通常更优。

一种相关的编码是one-cold编码，其中K个状态用K位表示，恰好其中一位为FALSE。

表3.8 三分频计数器的一位有效编码与二进制编码

State	One-Hot Encoding			Binary Encoding	
	S_2	S_1	S_0	S_1	S_0
S0	0	0	1	0	0
S1	0	1	0	0	1
S2	1	0	0	1	0

表3.9 二进制编码的状态转换表

Current State		Next State	
S_1	S_0	S'_1	S'_0
0	0	0	1
0	1	1	0
1	0	0	0

表3.10 独热编码的状态转换表

Current State			Next State		
S_2	S_1	S_0	S'_2	S'_1	S'_0
0	0	1	0	1	0
0	1	0	1	0	0
1	0	0	0	0	1

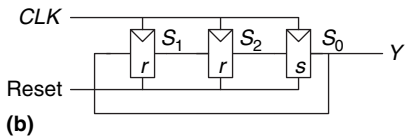
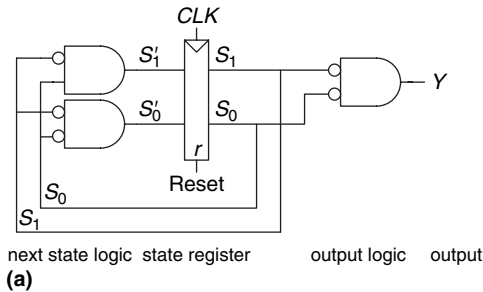
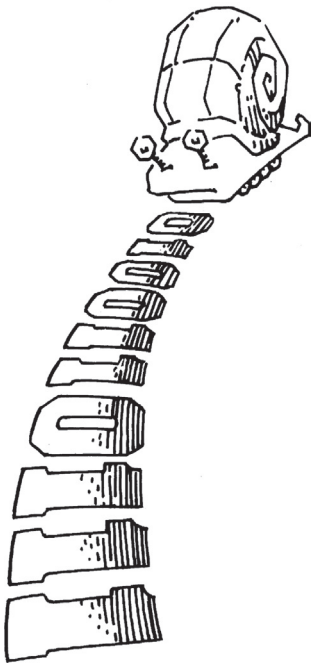


图3.29 除以3的电路：(a)二进制编码 (b)独热编码

记住两种有限状态机之间差异的一个简单方法是，对于给定的问题，摩尔机通常比米利机具有更多的*more*状态。

3.4.3 摩尔机和米利机

到目前为止，我们展示的都是摩尔机的示例，其输出仅取决于系统状态。因此，在摩尔机的状态转换图中，输出标注在圆圈内。回顾一下，米利机与摩尔机非常相似，但输出不仅取决于当前状态，还可能取决于输入。因此，在米利机的状态转换图中，输出标注在弧线上而非圆圈内。计算输出的组合逻辑模块使用当前状态和输入，如图3.22(b)所示。



示例3.7 摩尔机与米利机对比

Alyssa P. Hacker拥有一只宠物机器蜗牛，其大脑为有限状态机（FSM）。蜗牛沿着一条包含0和1序列的纸带从左向右爬行。每个时钟周期，蜗牛会爬向下一个比特位。当它爬过的最后两个比特为01时，蜗牛会微笑。请设计一个FSM来判断蜗牛何时应该微笑。输入A是蜗牛触角下方的比特位。输出Y在蜗牛微笑时为TRUE。比较摩尔机和米利机的状态机设计。针对每台机器，画出时序图，展示当Alyssa的蜗牛沿序列0100110111爬行时的输入、状态和输出。

解 摩尔型状态机需要三个状态，如图3.30(a)所示。请自行验证该状态转换图是否正确。特别地，为什么当输入为0时，存在一条从S2到S1的弧？

相比之下，Mealy机仅需两个状态，如图3.30(b)所示。每条弧线标记为A/Y。A是引起该转移的输入值，Y是对应的输出。

表3.11和表3.12分别展示了摩尔机的状态转换表和输出表。摩尔机至少需要两位状态。考虑使用二进制状态编码：S0 = 00，S1 = 01，S2 = 10。表3.13和表3.14分别用这些编码重写了状态转换表和输出表。

从这些表中，我们通过观察得到下一状态和输出方程。注意，这些方程利用了状态11不存在这一事实进行了简化。因此，对于不存在的状态，其对应的下一状态和输出为无关项（表中未显示）。我们利用这些无关项来最小化方程。

$$\begin{aligned} S'_1 &= S_0 A \\ S'_0 &= \overline{A} \end{aligned} \tag{3.8}$$

$$Y = S_1 \tag{3.9}$$

表3.15展示了Mealy机的组合状态转换与输出表。Mealy机仅需一位状态。考虑采用二进制状态编码：S0 = 0, S1 = 1。表3.16使用这些编码重写了状态转换与输出表。

从 在这些表中，我们通过观察得到下一状态和输出方程。 节。

$$S'_0 = \bar{A} \quad (3.10)$$

$$Y = S_0 A \quad (3.11)$$

摩尔和米利型机器的原理图如图3.31所示。每种机器的时序图如图3.32所示（见第133页）。这两种机器遵循不同的状态序列。此外，米利型机器的输出提前一个周期上升，因为它响应输入而非等待状态变化。如果米利型输出通过触发器延迟，它将与摩尔型输出匹配。在选择有限状态机设计风格时，需考虑您希望输出何时做出响应。

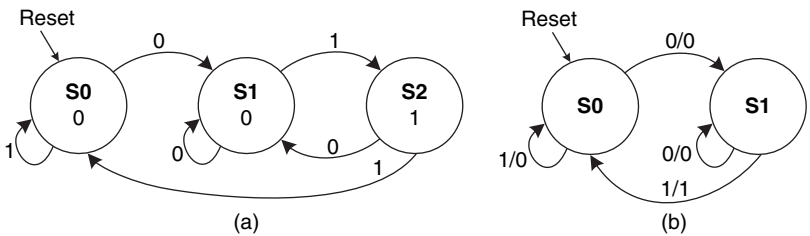


图3.30 FSM状态转换图：(a) Moore机，(b) Mealy机

表3.11 Moore状态转移表

Current State <i>S</i>	Input <i>A</i>	Next State <i>S'</i>
S0	0	S1
S0	1	S0
S1	0	S1
S1	1	S2
S2	0	S1
S2	1	S0

表3.12 Moore输出表

Current State <i>S</i>	Output <i>Y</i>
S0	0
S1	0
S2	1

表3.13 带有状态编码的摩尔状态转换表

Current State		Input	Next State	
S_1	S_0		S'_1	S'_0
0	0	0	0	1
0	0	1	0	0
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0

表3.14 带状态编码的Moore输出表

Current State		Output
S_1	S_0	
0	0	0
0	1	0
1	0	1

表3.15 Mealy状态转换与输出表

Current State	Input	Next State	Output
S		S'	Y
S0	0	S1	0
S0	1	S0	0
S1	0	S1	0
S1	1	S0	1

表3.16 带有状态编码的Mealy状态转换与输出表

Current State	Input	Next State	Output
S_0		S'_0	Y
0	0	1	0
0	1	0	0
1	0	1	0
1	1	0	1

3.4.4 状态机分解

设计复杂的FSM时，如果能够将其分解为多个相互交互的简单状态机，使得某些机器的输出成为其他机器的输入，通常会更容易。这种层次化和模块化的应用称为状态机的factoring。

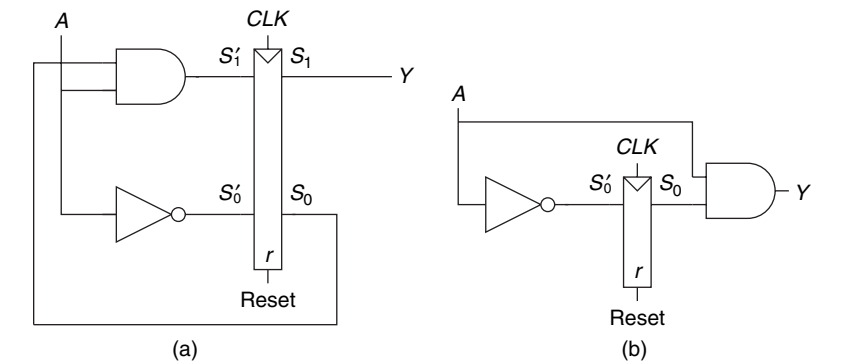


图3.31 (a) Moore机和(b) Mealy机的FSM示意图

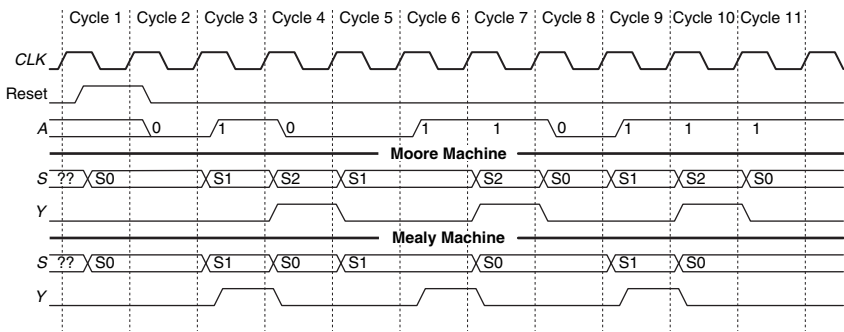


图3.32 Moore和Mealy机器的时序图

E示例3.8 未分解与分解状态机

NES

修改第3.4.1节的交通灯控制器，增加游行模式。当观众和乐队分散成群前往足球比赛时，该模式保持Bravado Boulevard绿灯亮起。控制器新增两个输入： P 和 R 。将 P 置为有效至少一个周期可进入游行模式，将 R 置为有效至少一个周期可退出游行模式。在游行模式下，控制器按正常序列运行直至 L_B 变为绿灯，随后保持该状态使 L_B 绿灯常亮，直至游行模式结束。

首先，绘制单个FSM的状态转换图，如图3.33(a)所示。然后，绘制两个交互FSM的状态转换图，如图3.33(b)所示。Mode FSM在处于游行模式时断言输出 M 。Lights FSM根据 M 以及交通传感器 T_A 和 T_B 控制灯光。

解 图3.34(a)展示了单一FSM设计。状态 S_0 至 S_3 处理正常模式，状态 S_4 至 S_7 处理游行模式。图的两半几乎相同，但在游行模式下，FSM停留在 S_6 ，此时Bravado Blvd方向亮绿灯。 P 和 R 输入控制这两半之间的切换。该FSM设计繁琐且杂乱。图3.34(b)展示了解分解后的FSM设计。模式FSM有两个状态，用于跟踪交通灯处于正常模式还是游行模式。灯光FSM经过修改，在 M 为真时停留在 S_2 。

图3.33 (a) 单设计 (b) 改进型
交通灯控制器FSM的因子设计

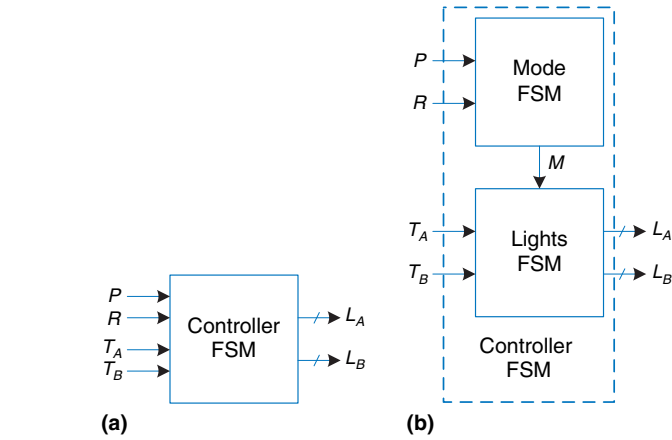
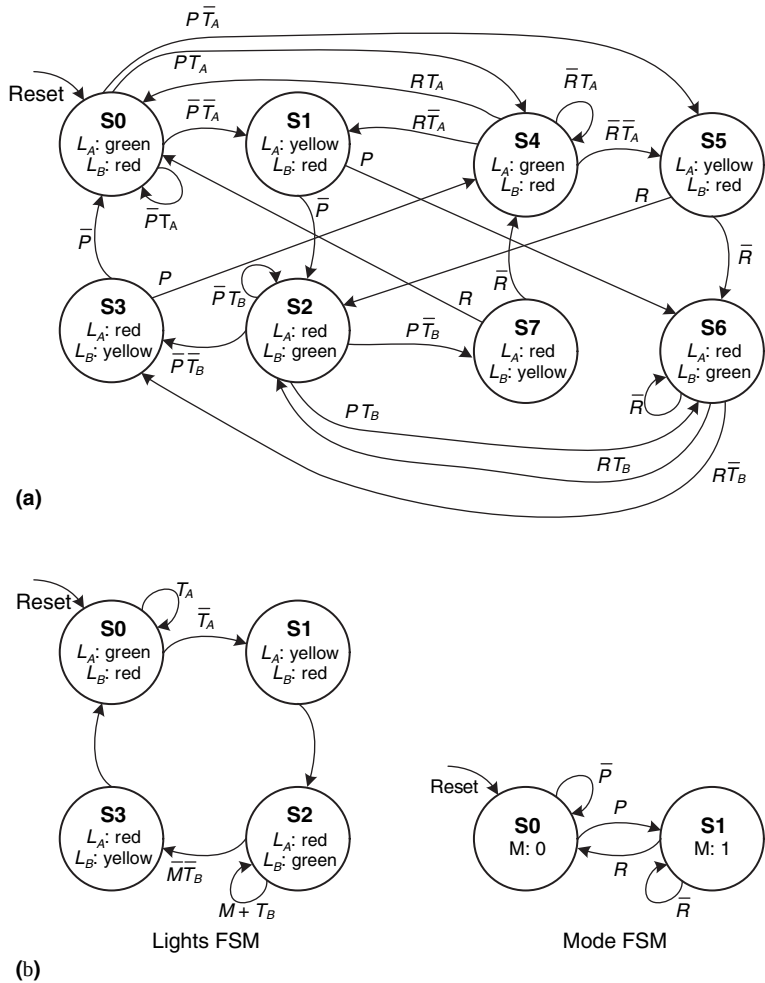


图3.34 状态转移
示意图: (a) 未分解, (b)
已分解



3.4.5 从原理图推导FSM

从原理图推导状态转换图的过程几乎与FSM设计相反。例如，在接手一个文档不完整的项目或逆向工程他人的系统时，这一过程可能是必要的。

检查电路，说明输入、输出和状态位。► 写出下一状态和输出方程。► 创建下一状态和输出表。► 简化下一状态表以消除不可达状态。► 为每个有效状态位组合分配名称。► 用状态名称重写下一状态和输出表。► 绘制状态转换图。► 用文字描述该有限状态机的功能。

在最后一步中，请注意简洁地描述FSM的整体目的和功能，而不要简单地复述状态转换图中的每个转换。

示例3.9 从电路推导FSM

Alyssa P. Hacker回到家，发现她的键盘锁被重新布线，旧密码已无法使用。锁上贴着一张纸条，上面是图3.35中的电路图。Alyssa认为这个电路可能是一个有限状态机，于是决定推导出状态转换图，看看能否帮她打开门。

解决方案 Alyssa 首先检查电路。输入为 $A_{1:0}$ ，输出为 *Unlock*。状态位已在图 3.35 中标注。这是一个 Moore

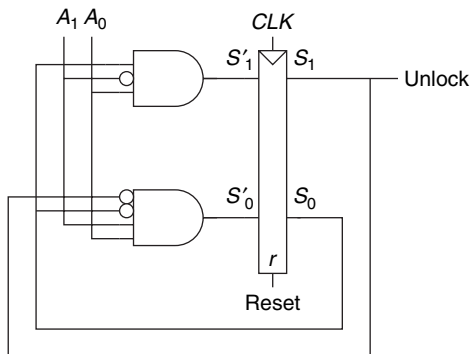


图3.35 示例3.9中找到的FSM电路

由于输出仅取决于状态位，因此从电路中，她直接写出下一状态和输出方程：

$$\begin{aligned} S_1' &= S_0 \overline{A_1} A_0 \\ S_0' &= \overline{S_1} \overline{S_0} A_1 A_0 \\ \text{Unlock} &= S_1 \end{aligned}$$

(3.12)

接下来，她根据方程写出下一状态表和输出表，分别如表3.17和表3.18所示，首先按照方程3.12的指示在表中填入1，其余位置填入0。

Alyssa 通过移除未使用的状态并利用无关项合并行来简化表格。在表 3.17 中， $S_{1:0} = 11$ 状态从未被列为可能的下一状态，因此移除了具有该当前状态的行。对于当前状态 $S_{1:0} = 10$ ，下一

表3.17 由图3.35电路导出的次态表

Current State		Input		Next State	
S_1	S_0	A_1	A_0	S_1'	S_0'
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	0	1	0	0
1	0	1	0	0	0
1	0	1	1	0	0
1	1	0	0	0	0
1	1	0	1	1	0
1	1	1	0	0	0
1	1	1	1	0	0

表3.18 由图3.35电路导出的输出表

Current State		Output
S_1	S_0	Unlock
0	0	0
0	1	0
1	0	1
1	1	1

表3.19 简化后的次态表

Current State		Input		Next State	
S_1	S_0	A_1	A_0	S'_1	S'_0
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	1	0	0
1	0	X	X	0	0

表3.20 简化输出表

Current State		Output
S_1	S_0	<i>Unlock</i>
0	0	0
0	1	0
1	0	1

表3.21 符号化下一状态表

Current State	Input	Next State
S	A	S'
S0	0	S0
S0	1	S0
S0	2	S0
S0	3	S1
S1	0	S0
S1	1	S2
S1	2	S0
S1	3	S0
S2	X	S0

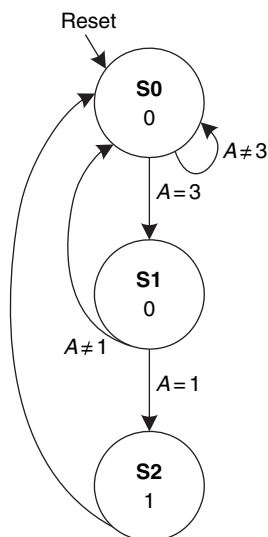
表3.22 符号输出表

Current State	Output
S	<i>Unlock</i>
S0	0
S1	0
S2	1

状态始终为 $S_{1:0} = 00$ ，与输入无关，因此对输入插入了无关项。简化后的表格如表3.19和表3.20所示。

她为每个状态位组合分配名称：S0 对应 $S_{1:0} = 00$ ，S1 对应 $S_{1:0} = 01$ ，S2 对应 $S_{1:0} = 10$ 。表3.21和表3.22分别展示了带有状态名称的次态表和输出表。

图3.36 示例3.9中FSM的状态转换图



Alyssa 使用表 3.21 和 3.22 记下了图 3.36 所示的状态转换图。通过观察，她发现有限状态机只有在检测到输入值 $A_{1:0}$ 为 3 后紧接着输入值为 1 时才会解锁门。随后门会再次锁上。Alyssa 在门键盘上尝试了这段代码，门打开了！

3.4.6 FSM 回顾

有限状态机是一种从书面规范中系统设计时序电路的有效方法。使用以下步骤设计一个 FSM：

识别输入和输出。► 绘制状态转换图。► 对于摩尔机：- 编写状态转换表。- 编写输出表。► 对于米利机：- 编写组合状态转换与输出表。► 选择状态编码——您的选择会影响硬件设计。► 编写下一状态和输出逻辑的布尔方程。► 绘制电路原理图。

本书将反复使用FSM来设计复杂的数字系统。

3.5 时序逻辑的时序

回想一下，触发器在时钟上升沿将输入 D 复制到输出 Q 。这一过程称为时钟边沿上的*sampling* D 。如果时钟上升时 D 稳定在0或1，则此行为是明确界定的。但若 D 在时钟上升的同时发生变化，会发生什么？

此问题类似于相机拍照时面临的场景。想象一下拍摄一只青蛙从荷叶跳入湖中的画面。如果你在跳跃前拍摄，会看到荷叶上的青蛙；如果在跳跃后拍摄，会看到水中的涟漪；但若在青蛙起跳的瞬间拍摄，可能会看到青蛙从荷叶延伸至水中的模糊影像。相机的特性由其 *aperture time* 决定，在此期间物体必须保持静止才能捕捉到清晰图像。同理，时序元件在时钟边沿附近存在一个 *aperture time*，在此期间输入必须保持稳定，触发器才能产生明确的输出。



时序元件的孔径由时钟沿前后的*setup*时间和*hold*时间定义。正如静态约束限制我们只能在禁止区外使用逻辑电平一样，*dynamic discipline* 限制我们只能在孔径时间外使用变化的信号。通过利用动态约束，我们可以将时间视为称为时钟周期的离散单位，就像我们将信号电平视为离散的1和0一样。信号可能在有限时间内出现毛刺并剧烈振荡。在动态约束下，我们只关心其在时钟周期结束时稳定后的最终值。因此，我们可以简单地写成 $A[n]$ ，即信号 A 在第 n th个时钟周期结束时的值，其中 n 是整数，而不是 $A(t)$ ，即信号 A 在某个时刻 t 的值，其中 t 是任意实数。

时钟周期必须足够长，以确保所有信号稳定。这限制了系统的速度。在实际系统中，时钟并非同时精确到达所有触发器。这种时间上的变化称为时钟偏移，进一步增加了所需的时钟周期。

有时无法满足动态约束，尤其是在与真实世界交互时。例如，考虑一个带有按钮输入的电路。猴子可能在时钟上升时按下按钮。这会导致一种称为*metastability*的现象，此时触发器捕获一个介于0和1之间的值，可能需要无限长的时间才能解析为有效的逻辑值。解决此类异步输入的方法是使用同步器，它产生非法逻辑值的概率非常小（但非零）。

我们在本节剩余部分详细阐述所有这些概念。

3.5.1 动态纪律

到目前为止，我们主要关注时序电路的功能规范。回顾一下，同步时序电路（如触发器或有限状态机）也具有时序规范，如图3.37所示。当时钟上升沿到来时，输出可能在时钟到 Q contamination delay、 t_{ccq} 之后开始变化，并且必须在时钟到 Q propagation delay、 t_{pcq} 之内稳定到最终值。这些分别代表了电路中最快和最慢的延迟。为了使电路正确采样其输入，输入必须在时钟上升沿之前至少稳定 $setup\ time$ 、 t_{setup} 的时间，并且在时钟上升沿之后至少保持稳定 $hold\ time$ 、 t_{hold} 的时间。建立时间和保持时间之和称为电路的 $aperture\ time$ ，因为这是输入必须保持稳定的总时间。

*dynamic discipline*指出，同步时序电路的输入必须在时钟边沿附近的建立与保持窗口时间内保持稳定。通过施加这一要求，我们确保触发器在信号未变化时对其进行采样。由于我们只关心输入在采样时刻的最终值，因此可以将信号视为在时间上和逻辑电平上均为离散的。

3.5.2 系统时序

在作者之一家庭购买Apple II+电脑至今的三十年间，微处理器时钟频率从1 MHz提升至数GHz，增幅超过1000倍。这一速度提升部分解释了计算机给社会带来的革命性变革。

*clock period*或*cycle time*，即 T_c ，是重复时钟信号上升沿之间的时间。其倒数 $f_c = 1/T_c$ ，即为*clock frequency*。在其他条件相同的情况下，提高时钟频率可以增加数字系统单位时间内完成的工作量。频率以赫兹（Hz）或每秒周期数为单位进行测量：1兆赫（MHz）= 10^6 Hz，1吉赫（GHz）= 10^9 Hz。

图3.38(a)展示了一个同步时序电路中的通用路径，我们需计算其时钟周期。在时钟上升沿，寄存器R1产生输出Q1。这些信号进入组合逻辑模块，生成D2，即寄存器R2的输入。图3.38(b)中的时序图显示，每个输出信号可能在污染延迟后开始变化。

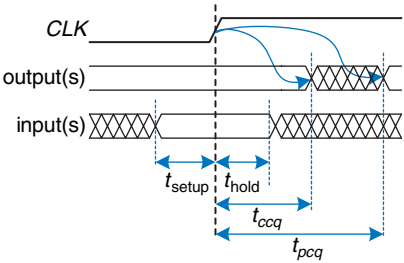


图3.37 同步时序电路的时序规范

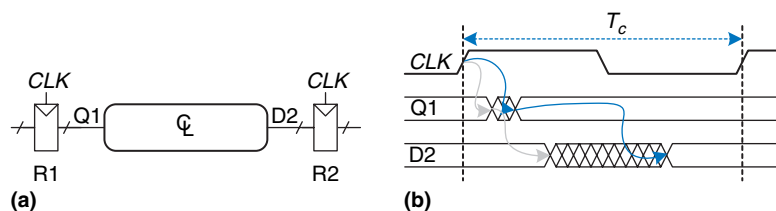


图3.38 寄存器之间的路径与时序图

输入变化并在其输入稳定后的传播延迟内稳定到最终值。灰色箭头表示通过R1和组合逻辑的污染延迟，蓝色箭头表示通过R1和组合逻辑的传播延迟。我们针对第二个寄存器R2的建立时间和保持时间分析时序约束。

建立时间约束

图3.39是仅显示路径最大延迟的时序图，该延迟由蓝色箭头标出。为满足R2的建立时间，D2必须在不晚于下一个时钟沿前的建立时间时稳定。因此，我们得到最小时钟周期的方程：

$$T_c \geq t_{pcq} + t_{pd} + t_{\text{setup}} \quad (3.13)$$

在商业设计中，时钟周期通常由工程总监或市场部门决定（以确保产品具有竞争力）。此外，触发器的时钟到Q传播延迟和建立时间 t_{pcq} 、 t_{setup} 由制造商指定。因此，我们重新排列公式3.13，以求解通过组合逻辑的最大传播延迟，这通常是单个设计师唯一可控的变量。

$$t_{pd} \leq T_c - (t_{pcq} + t_{\text{setup}}) \quad (3.14)$$

括号中的术语 $t_{pcq} + t_{\text{setup}}$ 被称为*sequencing over-head*。理想情况下，整个周期时间 T_c 可用于有效

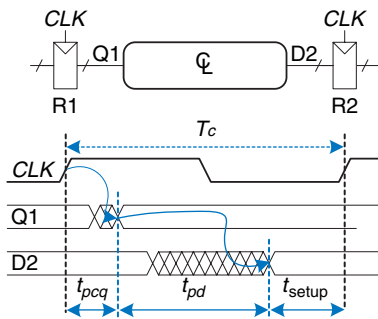


图3.39 建立时间约束的最大延迟

在组合逻辑中计算, t_{pd} 。然而, 触发器的时序开销会占用这部分时间。公式3.14被称为*setup time constraint*或*max-delay constraint*, 因为它取决于建立时间, 并限制了通过组合逻辑的最大延迟。

如果通过组合逻辑的传播延迟过大, D2可能无法在R2需要其稳定并进行采样时达到最终值。因此, R2可能会采样到错误的结果, 甚至是非法逻辑电平(即禁止区域内的电平)。在这种情况下, 电路将发生故障。该问题可以通过增加时钟周期或重新设计组合逻辑以缩短传播延迟来解决。

保持时间约束

图3.38(a)中的寄存器R2也有一个*hold time constraint*。其输入D2在时钟上升沿之后的某个时间 t_{hold} 之前不得改变。根据图3.40, D2可能在时钟上升沿之后 $t_{ccq} + t_{cd}$ 时立即改变。因此, 我们得到

$$t_{ccq} + t_{cd} \geq t_{hold} \quad (3.15)$$

同样地, t_{ccq} 和 t_{hold} 是触发器的特性, 通常不在设计者的控制范围内。通过重新整理, 我们可以求解组合逻辑的最小污染延迟:

$$t_{cd} \geq t_{hold} - t_{ccq} \quad (3.16)$$

方程3.16被称为*hold time constraint*或*min-delay constraint*, 因为它限制了通过组合逻辑的最小延迟。

我们假设任何逻辑元件都可以相互连接, 而不会引入时序问题。特别地, 我们期望两个触发器可以直接级联, 如图3.41所示, 而不会导致保持时间问题。

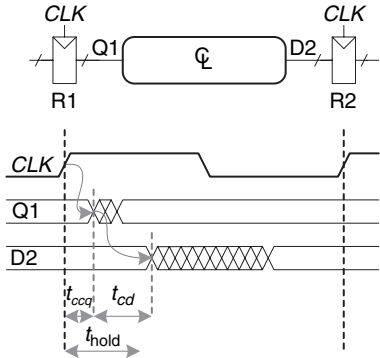


图3.40 保持时间约束的最小延迟

在这种情况下， $t_{cd} = 0$ ，因为触发器之间没有组合逻辑。代入公式3.16得到要求：

$$t_{hold} \leq t_{ccq} \tag{3.17}$$

换句话说，一个可靠的触发器必须使其保持时间短于其污染延迟。通常，触发器设计时令 $t_{hold} = 0$ ，从而始终满足公式3.17。除非另有说明，本书通常将采用这一假设，并忽略保持时间约束。

然而，保持时间约束至关重要。如果违反这些约束，唯一的解决方案是增加逻辑路径中的污染延迟，这需要重新设计电路。与建立时间约束不同，保持时间约束无法通过调整时钟周期来修复。在当今先进技术下，重新设计集成电路并制造修正后的设计需要数月时间和数百万美元，因此必须极其严肃地对待*hold time violations*。

综合运用

时序电路具有建立时间和保持时间约束，这些约束决定了触发器之间组合逻辑的最大和最小延迟。现代触发器通常设计为组合逻辑的最小延迟可以为0——即触发器可以背靠背放置。最大延迟约束限制了高速电路中关键路径上连续门电路的数量，因为高时钟频率意味着较短的时钟周期。

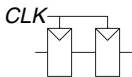


图3.41 背对背触发器

示例3.10 时序分析

本·比特迪德尔设计了图3.42中的电路。根据他所元件的数据手册，触发器具有30 ps的时钟到Q污染延迟和80 ps的传播延迟。它们的建立时间为50 ps，保持时间为60 ps。每个逻辑门的传播延迟为40 ps。

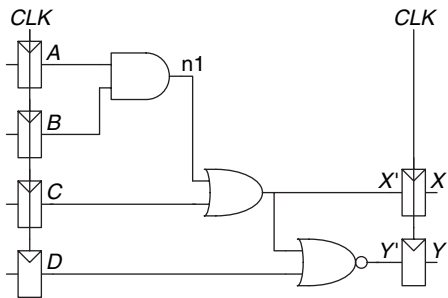


图3.42 时序分析示例电路

以及25 ps的污染延迟。帮助Ben确定最大时钟频率以及是否可能发生任何保持时间违规。这个过程称为*timing analysis*。

解决方案 图3.43(a)展示了信号可能发生变化时的波形。输入信号A至D是寄存的，因此它们仅在CLK上升后不久发生变化。

关键路径出现在B = 为1, C = 为0, D = 为0, 且A从0上升至1时，触发n1上升、X'上升、Y'下降，如图3.43(b)所示。该路径包含三个门延迟。对于关键路径，我们假设每个门都需要其完整的传播延迟。Y'必须在下一个CLK上升沿之前建立。因此，最小周期时间为

$$T_c \geq t_{pcq} + 3t_{pd} + t_{setup} = 80 + 3 \times 40 + 50 = 250 \text{ ps} \quad (3.18)$$

最大时钟频率为 $f_c = 1/T_c = 4 \text{ GHz}$ 。

当A = 为0且C上升时，会导致X'上升，从而产生短路径，如图3.43(c)所示。对于短路径，我们假设每个门仅在污染延迟后切换。该路径仅涉及一个门延迟，因此可能在 $t_{ccq} + t_{cd} = 30 + 25 = 55 \text{ ps}$ 后发生。但请记住，触发器的保持时间为

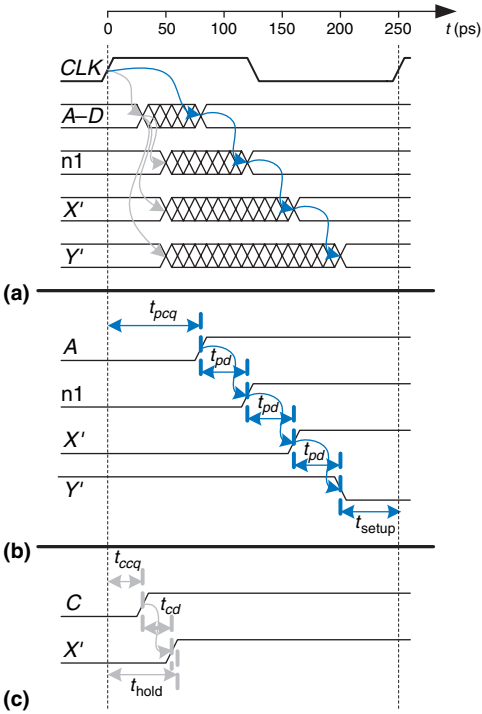


图3.43 时序图：(a) 一般情况，
(b) 关键路径，(c) 短路径

问题。然而，一些高性能微处理器（包括奔腾4）使用一种称为*pulsed latch*的元件来代替触发器。脉冲锁存器的行为类似于触发器，但具有较短的时钟到Q延迟和较长的保持时间。通常，添加缓冲器通常可以（但并非总是）解决保持时间问题，而不会减慢关键路径。

3.5.3 时钟偏移*

在前面的分析中，我们假设时钟同时到达所有寄存器。实际上，这一时间存在一定变化。这种时钟边沿的变化称为*clock skew*。例如，从时钟源到不同寄存器的导线长度可能不同，导致延迟略有差异，如图3.46所示。噪声也会导致不同的延迟。第3.2.5节描述的时钟门控会进一步延迟时钟。如果某些时钟被门控而其他未被门控，则门控时钟与非门控时钟之间将存在显著偏差。在图3.46中，CLK2相对于CLK1存在*early*，因为两个寄存器之间的时钟线绕了远路。如果时钟布线方式不同，CLK1可能反而提前。进行时序分析时，我们考虑最坏情况，以确保电路在所有条件下都能正常工作。

图3.47在图3.38的时序图中增加了偏斜。粗时钟线表示时钟信号可能到达任何寄存器的最晚时间；阴影线显示时钟可能提前最多 t_{skew} 到达。

首先，考虑图3.48所示的建立时间约束。在最坏情况下，R1接收最晚的偏斜时钟，而R2接收最早的偏斜时钟，从而为数据在寄存器之间传播留下尽可能少的时间。

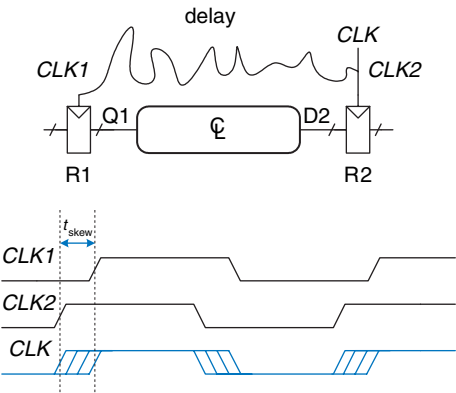


图3.46 由导线延迟引起的时钟偏移

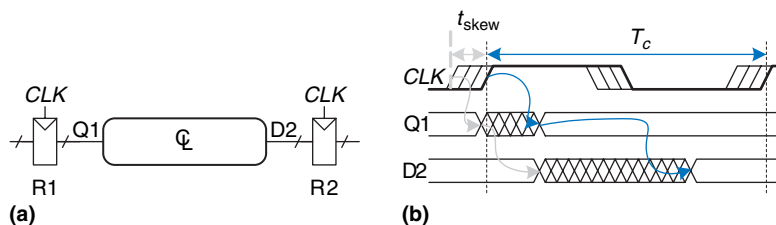


图3.47 具有时钟偏移的时序图

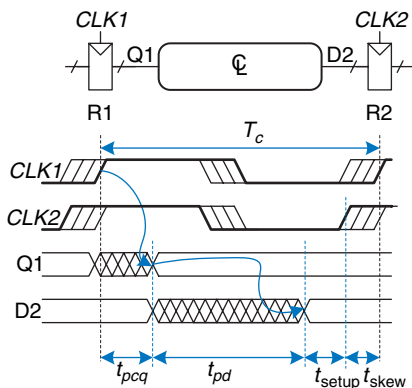


图3.48 带时钟偏斜的建立时间约束

数据通过寄存器和组合逻辑传播，并且必须在R2采样之前建立稳定。因此，我们得出结论：

$$T_c \geq t_{pcq} + t_{pd} + t_{setup} + t_{skew} \quad (3.19)$$

$$t_{pd} \leq T_c - (t_{pcq} + t_{setup} + t_{skew}) \quad (3.20)$$

接下来，考虑图3.49所示的保持时间约束。在最坏情况下，R1接收一个提前偏移的时钟CLK1，R2接收一个延迟偏移的时钟CLK2。数据快速通过寄存器和组合逻辑，但必须在延迟时钟之后的一个保持时间内才能到达。因此，我们得到

$$t_{ccq} + t_{cd} \geq t_{hold} + t_{skew} \quad (3.21)$$

$$t_{cd} \geq t_{hold} + t_{skew} - t_{ccq} \quad (3.22)$$

总之，时钟偏移实际上同时增加了建立时间和保持时间。它增加了时序开销，减少了组合逻辑中可用于有效工作的时间。同时，它也增加了组合逻辑所需的最小延迟。即使 $t_{hold} = 0$ ，如果 $t_{skew} > t_{ccq}$ ，一对背靠背的触发器也会违反公式3.22。为了防止严重的保持时间故障，设计者不得允许过多的

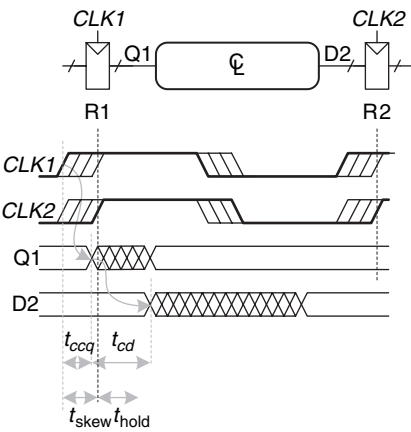


图3.49 带时钟偏移的保持时间约束

时钟偏移。有时，触发器会被有意设计得特别慢（即较大的 t_{ccq} ），以防止即使在时钟偏移显著时出现保持时间问题。

示例3.12 含时钟偏差的时序分析

重新审视示例3.10，并假设系统存在50 ps的时钟偏差。

解 关键路径保持不变，但建立时间因时钟偏移而有效增加。因此，最小周期时间为

$$\begin{aligned} T_c &\geq t_{pcq} + 3t_{pd} + t_{\text{setup}} + t_{\text{skew}} \\ &= 80 + 3 \times 40 + 50 + 50 = 300\text{ps} \end{aligned} \tag{3.23}$$

最大时钟频率为 $f_c = 1/T_c = 3.33\text{ GHz}$ 。

短路径仍保持55 ps不变。保持时间因时钟偏移有效增加至 $60 + 50 = 110\text{ ps}$ ，远大于55 ps。因此，该电路将在任何频率下违反保持时间并发生故障。即使没有时钟偏移，电路也已违反保持时间约束。系统中的时钟偏移只会使违规情况更加严重。

示例3.13 修复保持时间违例

转速假设示例3.11中系统具有50 ps的时钟 倾斜。

解决方案 关键路径不受影响，因此最大时钟频率仍为3.33 GHz。

短路径增加到80皮秒。这仍然小于 $t_{\text{hold}} + t_{\text{skew}} = 110$ 皮秒，因此电路仍然违反其保持时间约束。

为了解决这个问题，可以插入更多缓冲器。缓冲器也需要添加到关键路径上，这会降低时钟频率。或者，可以使用具有更短保持时间的更好的触发器。

3.5.4 亚稳态

如前所述，无法始终保证时序电路在孔径时间内输入稳定，尤其当输入来自外部世界时。考虑一个连接到触发器输入的按钮，如图3.50所示。当按钮未按下时， $D = 0$ 。当按钮按下时， $D = 1$ 。一只猴子在相对于 CLK 上升沿的随机时刻按下按钮。我们想知道 CLK 上升沿之后的输出 Q 。在情况I中，当按钮在 CLK 之前很久按下时， $Q = 1$ 。在情况II中，当按钮在 CLK 之后很久才按下时， $Q = 0$ 。但在情况III中，当按钮在 t_{setup} 之前到 CLK 之后的某个时间按下时，输入违反了动态规则，输出未定义。

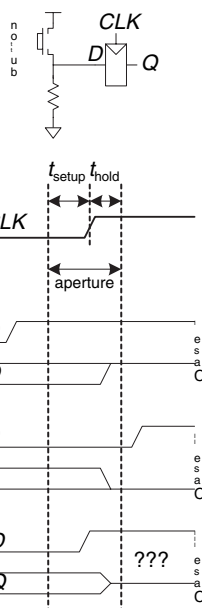


图3.50 光圈之前、之后或期间输入变化

亚稳态

当触发器在其孔径时间内对正在变化的输入进行采样时，输出 Q 可能会暂时处于0到 V_{DD} 之间的禁止区电压。这被称为*metastable*状态。最终，触发器会将输出解析为0或1的*stable state*。然而，达到稳定状态所需的*resolution time*是无界的。

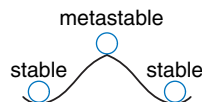


图3.51 稳定态与亚稳态

触发器的亚稳态类似于位于两个山谷之间山顶上的球，如图3.51所示。两个山谷是稳定状态，因为山谷中的球只要不受干扰就会停留在那里。山顶被称为亚稳态，因为如果球完全平衡，它会停留在那里。但由于没有什么是完美的，球最终会滚向一侧或另一侧。这种变化发生所需的时间取决于球最初平衡的程度。每个双稳态器件在两个稳定状态之间都有一个亚稳态。

解决时间

如果触发器输入在时钟周期内的随机时刻发生变化，则解析到稳定状态所需的解析时间 t_{res} 也是一个随机变量。如果输入在孔径之外发生变化，则 $t_{\text{res}} = t_{\text{pcq}}$ 。但如果输入恰好在孔径内发生变化， t_{res} 可能会显著延长。理论和实验分析（见第3.5.6节）已



表明, 解析时间 t_{res} 超过任意时间 t 的概率随 t 呈指数衰减:

$$P(t_{res} > t) = \frac{T_0}{T_c} e^{-\frac{t}{\tau}} \quad (3.24)$$

其中 T_c 是时钟周期, T_0 和 τ 是触发器的特性。该方程仅在 t 远大于 t_{pcq} 时成立。

直观上, T_0/T_c 描述了输入在不良时间(即孔径时间内)发生变化的概率; 该概率随周期时间 T_c 的增加而减小。 τ 是一个时间常数, 表示触发器离开亚稳态的速度; 它与触发器中交叉耦合门的延迟有关。

总之, 如果触发器等双稳态器件的输入在孔径时间内发生变化, 其输出可能会在一段时间内处于亚稳态值, 随后才稳定到确定的0或1。解决亚稳态所需的时间是无上限的, 因为在任何有限时间 t 内, 触发器仍处于亚稳态的概率都不为零。然而, 随着 t 的增加, 该概率呈指数级下降。因此, 如果我们等待足够长的时间(远长于 t_{pcq}), 就可以以极高的概率预期触发器将达到有效的逻辑电平。

3.5.5 同步器

数字系统从现实世界接收异步输入是不可避免的。例如, 人类输入就是异步的。如果处理不当, 这些异步输入可能导致系统内部出现亚稳态电压, 引发难以追踪和纠正的随机系统故障。数字系统设计者的目标应是确保在存在异步输入的情况下, 遇到亚稳态电压的概率足够小。“足够”取决于具体场景。对于手机而言, 或许每10年出现一次故障是可以接受的, 因为用户可以在死机时重启设备。而对于医疗设备, 宇宙预期寿命(10^{10} 年)内出现一次故障才是更理想的目标。为保证良好的逻辑电平, 所有异步输入都应通过synchronizers处理。

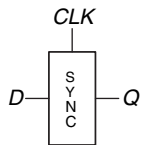


图3.52 同步器 符号

如图3.52所示, 同步器是一种接收异步输入 D 和时钟 CLK 的设备。它在有限时间内产生输出 Q , 且该输出具有极高概率的有效逻辑电平。若 D 在孔径时间内保持稳定, 则 Q 应取与 D 相同的值。若 D 在孔径时间内发生变化, 则 Q 可能取高电平或低电平, 但不得处于亚稳态。

图3.53展示了一种用两个触发器构建同步器的简单方法。F1在 CLK 的上升沿对 D 进行采样。如果此时 D 正在变化, 输出 $D2$ 可能会暂时处于亚稳态。如果时钟

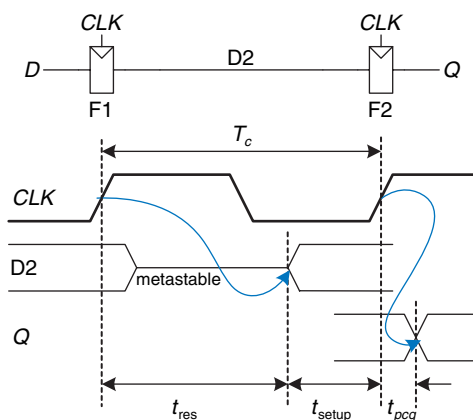


Figure 3.53 Simple synchronizer

如果周期足够长，D2 有很大概率在周期结束前稳定到一个有效的逻辑电平。随后 F2 对此时已稳定的 D2 进行采样，产生一个良好的输出 Q。

我们称同步器 *fails* 如果 Q，即同步器的输出，进入亚稳态。这种情况可能发生在 D2 未能在必须于 F2 建立之前解析为有效电平时——即如果 $t_{res} > T_c - t_{setup}$ 。根据公式 3.24，随机时刻单次输入变化导致失效的概率为

$$P(\text{failure}) = \frac{T_0}{T_c} e^{-\frac{T_c - t_{setup}}{\tau}} \quad (3.25)$$

故障概率 $P(\text{failure})$ 是指当 D 发生单次变化时，输出 Q 处于亚稳态的概率。若 D 每秒变化一次，则每秒的故障概率即为 $P(\text{failure})$ 。然而，若 D 每秒变化 N 次，每秒的故障概率将增大至 N 倍：

$$P(\text{failure})/\text{sec} = N \frac{T_0}{T_c} e^{-\frac{T_c - t_{setup}}{\tau}} \quad (3.26)$$

系统可靠性通常以 *mean time between failures* (MTBF) 来衡量。顾名思义，MTBF 是系统两次故障之间的平均时间。它是系统在任何给定秒内发生故障概率的倒数。

$$MTBF = \frac{1}{P(\text{failure})/\text{sec}} = \frac{T_c e^{\frac{T_c - t_{setup}}{\tau}}}{NT_0} \quad (3.27)$$

公式 3.27 表明，随着同步器等待时间 T_c 的增加，MTBF 呈指数级改善。对于大多数系统，同步器

等待一个时钟周期可提供安全的MTBF。在极高速度的系统中，可能需要等待更多周期。

示例3.14 FSM输入的同步器

第3.4.1节中的交通灯控制器FSM接收来自交通传感器的异步输入。假设使用同步器来确保输入到控制器的信号稳定。交通平均每秒到达0.2次。同步器中的触发器具有以下特性： $\tau = 200$ ps， $T_0 = 150$ ps，以及 $t_{\text{setup}} = 500$ ps。为使MTBF超过1年，同步器的时钟周期必须为多长？

解1年 $\approx \pi \times 10^7$ 秒。解方程3.27。

$$\pi \times 10^7 = \frac{T_c e^{\frac{T_c - 500 \times 10^{-12}}{200 \times 10^{-12}}}}{(0.2)(150 \times 10^{-12})} \quad (3.28)$$

该方程没有闭式解。不过，通过猜测和验证的方法很容易求解。在电子表格中，尝试几个 T_c 的值，并计算MTBF，直到找到使MTBF为1年的 T_c 值： $T_c = 3.036$ 纳秒。

3.5.6 分辨时间推导*

方程3.24可以利用电路理论、微分方程和概率的基本知识推导得出。如果你对推导过程不感兴趣，或不熟悉相关数学知识，可以跳过本节。

如果触发器在某个时间 t 后对变化的输入进行采样（导致亚稳态情况），并且输出在时钟沿后的该时间内未能解析为有效电平，则触发器输出将处于亚稳态。符号化地，这可以表示为

$$P(t_{\text{res}} > t) = P(\text{samples changing input}) \times P(\text{unresolved}) \quad (3.29)$$

我们分别考虑每个概率项。异步输入信号在某个时间 t_{switch} 内在0和1之间切换，如图3.54所示。在时钟边沿附近的开窗期间，输入发生变化的概率为

$$P(\text{samples changing input}) = \frac{t_{\text{switch}} + t_{\text{setup}} + t_{\text{hold}}}{T_c} \quad (3.30)$$

如果触发器确实进入亚稳态——即，以概率 $P(\text{采样变化输入})$ ——从亚稳态恢复的时间取决于电路的内部工作机制。这个恢复时间决定了 $P(\text{未恢复})$ ，即触发器尚未

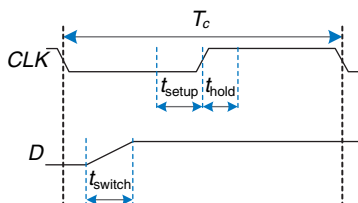


图3.54 输入时序

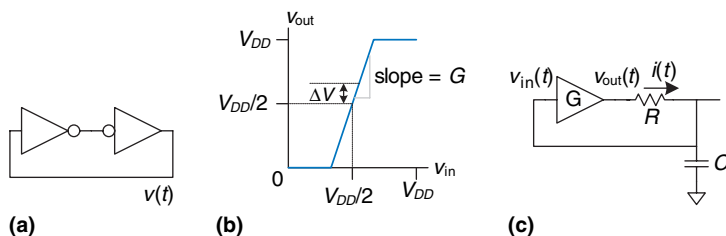


图3.55 双稳态器件的电路模型

在时间 t 后解析为有效逻辑电平。本节剩余部分将分析双稳态器件的简单模型，以估算该概率。

双稳态器件利用正反馈进行存储。图3.55(a)展示了一对反相器实现的这种反馈；该电路的行为代表了大多数双稳态元件。一对反相器的行为类似于缓冲器。我们将缓冲器建模为具有图3.55(b)所示的对称直流传输特性，其斜率为 G 。缓冲器只能提供有限的输出电流；我们可以将其建模为输出电阻 R 。所有实际电路也都存在必须充电的电容 C 。通过电阻对电容充电会导致RC延迟，从而阻止缓冲器瞬间切换。因此，完整的电路模型如图3.55(c)所示，其中 $v_{\text{out}}(t)$ 是表示双稳态器件状态的关键电压。

该电路的亚稳态点为 $v_{\text{out}}(t) = v_{\text{in}}(t) = V_{DD}/2$ ；如果电路恰好从该点开始，在没有噪声的情况下，它将无限期地保持在该点。由于电压是连续变量，电路恰好从亚稳态点开始的概率微乎其微。然而，电路可能在时间0从接近亚稳态的点开始，即 $v_{\text{out}}(0) = V_{DD}/2 + \Delta V$ ，其中存在一个小的偏移量 ΔV 。在这种情况下，如果 $\Delta V > 0$ ，正反馈最终会将 $v_{\text{out}}(t)$ 驱动至 V_{DD} ；如果 $\Delta V < 0$ ，则驱动至0。达到 V_{DD} 或0所需的时间即为双稳态器件的分辨时间。

DC传输特性是非线性的，但在亚稳态附近（即我们关注的区域）呈现线性。具体而言，若 $v_{\text{in}}(t) = V_{DD}/2 + \Delta V/G$ ，则对于小 ΔV 有 $v_{\text{out}}(t) = V_{DD}/2 + \Delta V$ 。流过电阻的电流为 $i(t) = (v_{\text{out}}(t) - v_{\text{in}}(t))/R$ 。

电容器以速率 $dv_{in}(t)/dt = i(t)/C$ 充电。综合这些事实，我们得到输出电压的控制方程。

$$\frac{dv_{out}(t)}{dt} = \frac{(G-1)}{RC} \left[v_{out}(t) - \frac{V_{DD}}{2} \right] \quad (3.31)$$

这是一个线性一阶微分方程。利用初始条件 $v_{out}(0) = V_{DD}/2 + \Delta V$ 求解得到

$$v_{out}(t) = \frac{V_{DD}}{2} + \Delta V e^{\frac{(G-1)t}{RC}} \quad (3.32)$$

图3.56绘制了给定不同起始点时 $v_{out}(t)$ 的轨迹。 $v_{out}(t)$ 从亚稳态点 $V_{DD}/2$ 呈指数远离，直到饱和于 V_{DD} 或 0。输出最终稳定为 1 或 0。这一过程所需的时间取决于与亚稳态点 ($V_{DD}/2$) 的初始电压偏移 (ΔV)。

求解方程 3.32 中的分辨率时间 t_{res} ，使得 $v_{out}(t_{res}) = V_{DD}$ 或 0，得到

$$|\Delta V| e^{\frac{(G-1)t_{res}}{RC}} = \frac{V_{DD}}{2} \quad (3.33)$$

$$t_{res} = \frac{RC}{G-1} \ln \frac{V_{DD}}{2|\Delta V|} \quad (3.34)$$

总之，若双稳态器件具有高电阻或高电容导致输出变化缓慢，则分辨率时间会增加；若双稳态器件具有高 *gain*、 G ，则分辨率时间会减少。此外，当电路起始状态越接近亚稳态点 ($\Delta V \rightarrow 0$) 时，分辨率时间会呈对数增长。

将 τ 定义为 $\frac{RC}{G-1}$ 。求解方程 3.34 中的 ΔV ，可得到初始偏移量 ΔV_{res} ，该偏移量对应特定的解析时间 t_{res} ：

$$\Delta V_{res} = \frac{V_{DD}}{2} e^{-t_{res}/\tau} \quad (3.35)$$

假设双稳态器件在输入变化时对其进行采样。它测量一个电压 $v_{in}(0)$ ，我们假设该电压在 0 到 V_{DD} 之间均匀分布。输出尚未

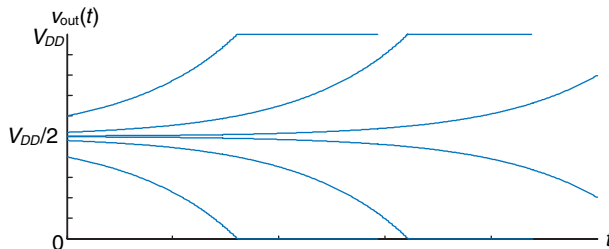


图3.56 分辨率轨迹

在时间 t_{res} 后解析为合法值的概率取决于初始偏移是否足够小。具体而言， v_{out} 上的初始偏移必须小于 ΔV_{res} ，因此 v_{in} 上的初始偏移必须小于 $\Delta V_{res}/G$ 。那么，双稳态器件在某一时刻对输入进行采样以获得足够小初始偏移的概率为

$$P(\text{unresolved}) = P\left(\left|v_{in}(0) - \frac{V_{DD}}{2}\right| < \frac{\Delta V_{res}}{G}\right) = \frac{2\Delta V_{res}}{GV_{DD}} \quad (3.36)$$

综合以上所述，解决时间超过某个时间点 t 的概率由以下方程给出：

$$P(t_{res} > t) = \frac{t_{switch} + t_{setup} + t_{hold}}{GT_c} e^{-\frac{t}{\tau}} \quad (3.37)$$

观察方程3.37符合方程3.24的形式，其中 $T_0 = (t_{switch} + t_{setup} + t_{hold})/G$ 且 $\tau = RC/(G - 1)$ 。总之，我们推导出了方程3.24，并展示了 T_0 和 τ 如何依赖于双稳态器件的物理特性。

3.6 并行性

系统的速度由信息在其中移动的延迟和吞吐量来表征。我们将 $token$ 定义为一组输入，经过处理后产生一组输出。这个术语让人联想到将地铁代币放在电路图上并移动它们，以可视化数据在电路中的流动。系统的 $latency$ 是一个代币从开始到结束通过系统所需的时间。 $throughput$ 是单位时间内可以产生的代币数量。

示例3.15 COOKIE吞吐量与延迟

本·比特迪德尔正在举办一场牛奶和饼干派对，庆祝他的交通灯控制器安装完成。他需要5分钟来擀好饼干并放在烤盘上，然后饼干在烤箱中烘烤需要15分钟。一旦饼干烤好，他就开始准备下一盘。请问本制作一盘饼干的吞吐量和延迟是多少？

解 在此示例中，一盘饼干是一个令牌。延迟为每盘1/3小时。吞吐量为3盘/小时。

正如您可能想象的那样，通过同时处理多个令牌可以提高吞吐量。这被称为*parallelism*，它有两种形式：空间和*spatial parallelism*，



提供多份硬件副本，以便同时执行多个任务。使用 *temporal parallelism*，任务被分解成多个阶段，如同流水线一般。多个任务可以分布在这些阶段中。尽管每个任务必须经过所有阶段，但在任何给定时间，每个阶段中都会有一个不同的任务，因此多个任务可以重叠进行。时间并行性通常被称为 *pipelining*。空间并行性有时简称为 *parallelism*，但我们将避免这种命名惯例，因为它容易产生歧义。

示例3.16 COOKIE并行性

Ben Bitdiddle 有数百位朋友要来参加他的派对，需要更快地烤制饼干。他正在考虑使用空间和/或时间并行性。

空间并行：Ben请Alyssa P. Hacker帮忙。她有自己的烤盘和烤箱。

时间并行：Ben拿到了第二个饼干托盘。一旦他将一个托盘放入烤箱，就开始在另一个托盘上擀饼干，而不是等待第一个托盘烤好。

使用空间并行时的吞吐量和延迟是多少？使用时间并行时呢？同时使用两者时呢？

解决方案 延迟是指从开始到完成一个任务所需的时间。在所有情况下，延迟均为1/3小时。如果Ben开始时没有饼干，那么延迟就是他必须等待直到吃到第一块饼干的时间。

吞吐量是每小时饼干盘的数量。采用空间并行时，Ben和Alyssa每人每20分钟完成一盘。因此，吞吐量翻倍，达到6盘/小时。采用时间并行时，Ben每15分钟将一盘新饼干放入烤箱，吞吐量为4盘/小时。这些情况如图3.57所示。

如果Ben和阿莉莎同时使用两种技术，他们每小时可以烤8盘。

考虑一个延迟为 L 的任务。在无并行系统中，吞吐量为 $1/L$ 。在具有 N 份硬件副本的空间并行系统中，吞吐量为 N/L 。在时间并行系统中，任务理想情况下被划分为 N 个等长的步骤或阶段。在这种情况下，吞吐量也为 N/L ，且仅需一份硬件副本。然而，正如饼干示例所示，找到 N 个等长的步骤通常不切实际。若最长步骤的延迟为 L_1 ，则流水线吞吐量为 $1/L_1$ 。

。

流水线（时间并行性）特别有吸引力，因为它能在不复制硬件的情况下加速电路。相反，寄存器被放置在组合逻辑块之间，以将逻辑划分为

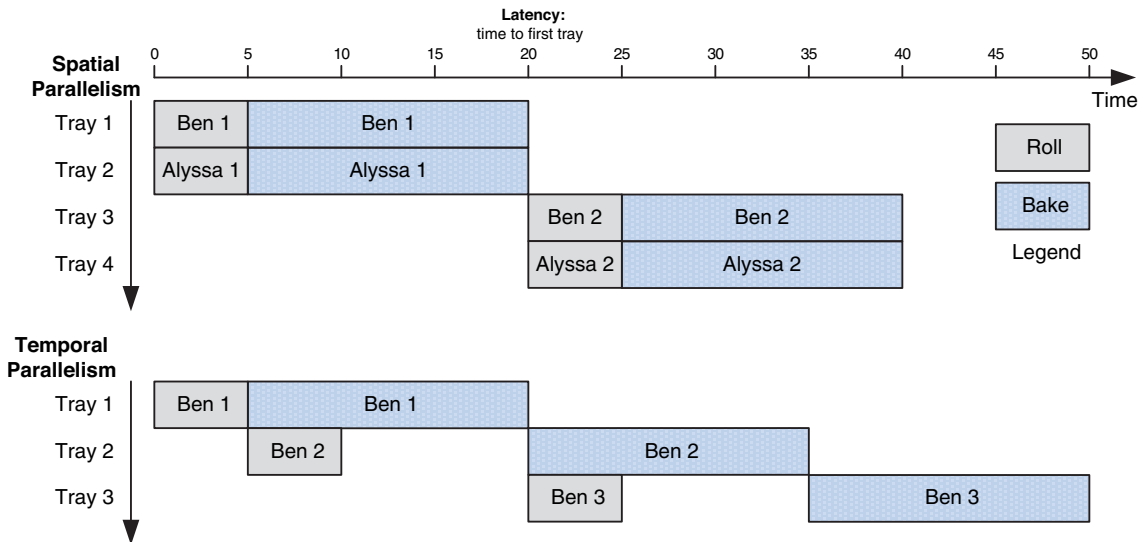


图3.57 饼干厨房中的空间与时间并行性

更短的阶段，可以以更快的时钟运行。寄存器防止一个流水线阶段中的令牌追上并破坏下一阶段中的令牌。

图3.58展示了一个无流水线电路的示例。寄存器之间包含四个逻辑块。关键路径经过块2、3和4。假设寄存器的时钟到Q传播延迟为0.3 ns，建立时间为0.2 ns。那么，周期时间为 $T_c = 0.3 + 3 + 2 + 4 + 0.2 = 9.5$ ns。该电路的延迟为9.5 ns，吞吐量为 $1/9.5 \text{ ns} = 105 \text{ MHz}$ 。

图3.59展示了同一电路通过在第3和第4模块之间添加寄存器，划分为一个两级流水线。第一级的最小时钟周期为 $0.3 + 3 + 2 + 0.2 = 5.5$ 纳秒。第二级的最小时钟周期为 $0.3 + 4 + 0.2 = 4.5$ 纳秒。时钟必须足够慢以使所有级都能正常工作。因此， $T_c = 5.5$ 纳秒。延迟为

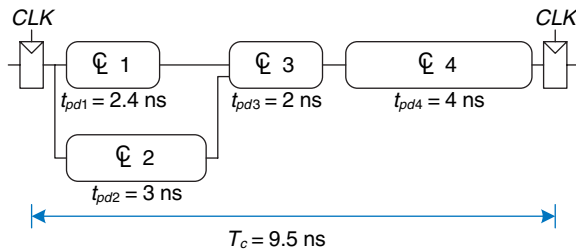


图3.58 无流水线电路

图3.59 两级流水线电路

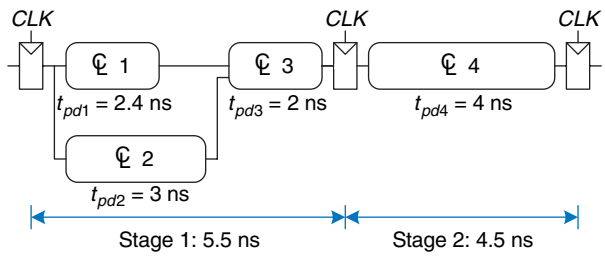
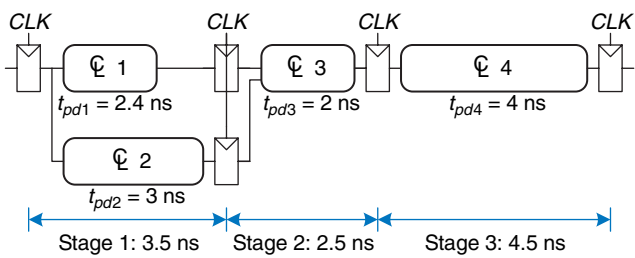


图3.60 三级流水线电路



两个时钟周期，即11纳秒。吞吐量为 $1/5.5 \text{ 纳秒} = 182 \text{ 兆赫}$ 。此例表明，在实际电路中，两级流水线几乎使吞吐量翻倍，同时略微增加了延迟。相比之下，理想流水线会在不增加延迟的情况下使吞吐量恰好翻倍。这种差异源于电路无法被精确分成两个完全相等的部分，且寄存器引入了更多的时序开销。

图3.60展示了同一电路被划分为三级流水线的情况。注意，需要额外两个寄存器来存储第一流水线阶段结束时模块1和模块2的结果。现在周期时间受限于第三阶段，为4.5纳秒。延迟为三个周期，即13.5纳秒。吞吐量为 $1/4.5 \text{ 纳秒} = 222 \text{ 兆赫}$ 。同样，增加流水线阶段以牺牲部分延迟为代价提高了吞吐量。

尽管这些技术功能强大，但并非适用于所有情况。并行处理的克星是dependencies。如果当前任务依赖于先前任务的结果，而非仅依赖于当前任务中的先前步骤，则该任务必须等待先前任务完成后才能开始。例如，如果本想在开始准备第二盘饼干之前先确认第一盘饼干味道不错，那么他就存在依赖关系，从而无法进行流水线或并行操作。并行处理是设计高性能数字系统最重要的技术之一。第7章将进一步讨论流水线技术，并展示处理依赖关系的示例。

3.7 总结

本章介绍了时序逻辑的分析与设计。与组合逻辑不同，组合逻辑的输出仅取决于当前输入，而时序逻辑的输出则同时取决于当前和先前的输入。换句话说，时序逻辑能够记住先前输入的信息。这种记忆被称为 *state of the logic*。

时序电路难以分析且容易设计错误，因此我们仅使用一组精心设计的构建模块。对我们而言最重要的元件是触发器，它接收时钟和输入 D ，并产生输出 Q 。触发器在时钟上升沿将 D 复制到 Q ，否则保持 Q 的旧状态。共享同一时钟的一组触发器称为 *register*。触发器也可能接收复位或使能控制信号。

任何能发明一种输出依赖于未来输入的逻辑的人，都会变得极其富有！

尽管存在多种形式的时序逻辑，但我们坚持使用同步时序电路，因为它们易于设计。同步时序电路由时钟寄存器分隔的组合逻辑块组成。电路的状态存储在寄存器中，并仅在时钟边沿更新。

有限状态机是设计时序电路的一种强大技术。设计FSM时，首先确定机器的输入和输出，并绘制状态转换图，标明状态及其之间的转换。选择状态编码，将状态图重写为状态转换表和输出表，根据当前状态和输入指示下一状态和输出。根据这些表格，设计用于计算下一状态和输出的组合逻辑，并绘制电路图。

同步时序电路具有时序规范，包括时钟到 Q 的传播延迟和污染延迟、 t_{pcq} 和 t_{ccq} ，以及建立时间和保持时间、 t_{setup} 和 t_{hold} 。为了正确运行，其输入必须在孔径时间内保持稳定，该孔径时间从时钟上升沿之前的建立时间开始，到时钟上升沿之后的保持时间结束。系统的最小周期时间 T_c 等于通过组合逻辑的传播延迟 t_{pd} 加上寄存器的 $t_{pcq} + t_{setup}$ 。为了正确运行，通过寄存器和组合逻辑的污染延迟必须大于 t_{hold} 。尽管存在普遍的误解，但保持时间并不影响周期时间。

系统整体性能通过延迟和吞吐量来衡量。延迟是指一个令牌从开始到结束所需的时间。吞吐量是系统在单位时间内能够处理的令牌数量。并行化能够提升系统的吞吐量。

练习3.5 给定图3.63所示的输入波形，画出D触发器的输出 Q 。

练习3.6 给定图3.64所示的输入波形，画出D触发器的输出 Q 。

练习3.7 图3.65中的电路是组合逻辑还是时序逻辑？请简要说明输入与输出之间的关系。你会如何称呼这个电路？

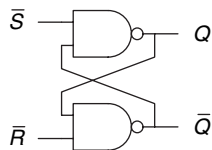


Figure 3.65 Mystery circuit

练习3.8 图3.66中的电路是组合逻辑还是时序逻辑？请简要说明输入与输出之间的关系。你会如何称呼这个电路？

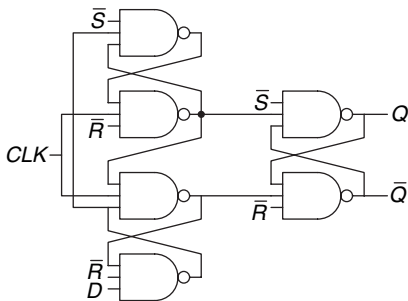


Figure 3.66 Mystery circuit

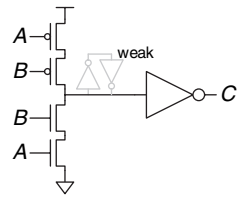
练习3.9 *toggle (T) flip-flop*有一个输入 CLK 和一个输出 Q 。在每个 CLK 的上升沿， Q 翻转为前一个值的补码。使用D触发器和反相器画出T触发器的原理图。

练习3.10 A *JK flip-flop*接收一个时钟和两个输入， J 和 K 。在时钟上升沿，它更新输出 Q 。如果 J 和 K 均为0，则 Q 保持其旧值。如果只有 J 为1，则 Q 变为1。如果只有 K 为1，则 Q 变为0。如果 J 和 K 均为1，则 Q 变为其当前状态的反值。

(a) 使用D触发器和一些组合逻辑构建一个JK触发器。 (b) 使用JK触发器和一些组合逻辑构建一个D触发器。 (c) 使用JK触发器构建一个T触发器（见习题3.9）。

练习3.11 图3.67中的电路称为Muller C-element。请简要说明输入与输出之间的关系。

图3.67 Muller C单元



练习3.12 设计一个异步复位的D锁存器，使用逻辑门实现。

练习3.13 设计一个异步可复位D触发器，使用逻辑门实现。

练习3.14 设计一个同步置位的D触发器，使用逻辑门实现。

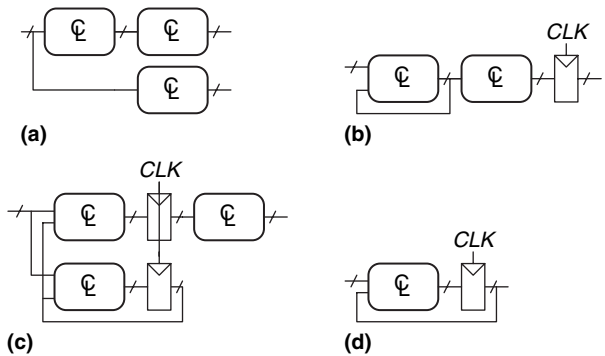
练习3.15 设计一个可异步置位的D触发器，使用逻辑门实现。

练习3.16 假设一个环形振荡器由N个反相器连接成环路构成。每个反相器的最小延迟为 t_{cd} ，最大延迟为 t_{pd} 。若N为奇数，请确定该振荡器可能工作的频率范围。

练习3.17 为什么在练习3.16中N必须是奇数？

练习3.18 图3.68中的哪些电路是同步时序电路？请解释。

图3.68 电路



练习3.19 你正在为一栋25层的大楼设计电梯控制器。控制器有两个输入：UP和DOWN。它产生一个输出，指示电梯所在的楼层。没有13层。控制器中状态的最小位数是多少？

练习3.20 你正在设计一个有限状态机（FSM），用于跟踪数字设计实验室中四名学生的`情绪`。每名学生的`情绪`可能是HAPPY（电路正常工作）、SAD（电路烧毁）、BUSY（正在处理电路）、CLUELESS（对电路感到困惑）或ASLEEP（趴在电路板上睡着了）。该FSM有多少个状态？表示这些状态所需的最少比特数是多少？

练习3.21 如何将练习3.20中的FSM分解为多个更简单的机器？每个更简单的机器有多少个状态？在这种分解设计中，所需的最小总比特数是多少？

练习3.22 用文字描述图3.69中的状态机功能。使用二进制状态编码，完成该有限状态机的状态转换表和输出表。写出下一状态和输出的布尔方程，并画出该有限状态机的原理图。

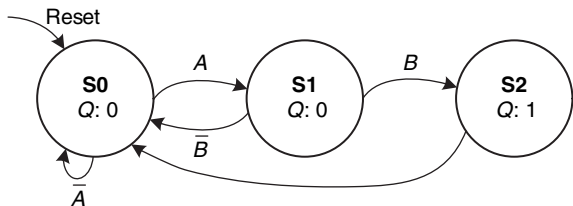


图3.69 习题3.22的状态转换图

练习3.23 用文字描述图3.70中的状态机功能。使用二进制状态编码，完成该有限状态机的状态转换表和输出表。写出下一状态和输出的布尔方程，并画出该有限状态机的原理图。

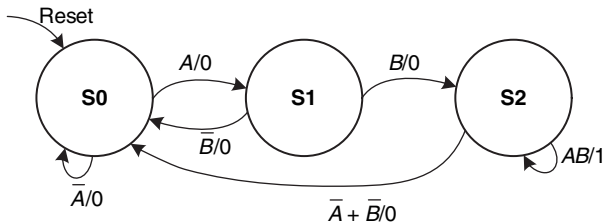


图3.70 习题3.23的状态转换图

练习3.24 学术大道与布拉瓦多大街的交叉口仍在发生事故。足球队在B灯变绿的那一刻冲进路口，与那些在A灯即将变绿前跌跌撞撞进入路口的睡眠不足的计算机专业学生相撞。

红灯。扩展第3.4.1节中的交通灯控制器，使得两个灯在任一灯再次变绿前先保持5秒的红灯。绘制改进后的摩尔机状态转换图、状态编码、状态转换表、输出表、下一状态和输出方程，以及你的FSM原理图。

练习3.25 来自第3.4.3节的Alyssa P. Hacker的蜗牛有一个采用Mealy机FSM大脑的女儿。每当她滑过模式1101或模式1110时，这只女儿蜗牛就会微笑。用尽可能少的状态画出这只快乐蜗牛的状态转换图。选择状态编码，并使用你的编码编写一个组合状态转换和输出表。写出下一状态和输出方程，并画出你的FSM原理图。

练习3.26 你被邀请为系休息室设计一台汽水自动售货机。汽水由IEEE学生分会部分补贴，因此每瓶仅需25美分。机器接受5美分、10美分和25美分硬币。当投入足够硬币时，机器会出汽水并找零。请为这台汽水机设计一个有限状态机控制器。FSM的输入为*Nickel*、*Dime*和*Quarter*，分别表示投入的硬币类型。假设每个周期仅投入一枚硬币。输出为*Dispense*、*ReturnNickel*、*ReturnDime*和*ReturnTwoDimes*。当FSM累计达到25美分时，它断言*Dispense*并输出必要的*Return*以完成找零。随后，它应准备好开始接收下一瓶汽水的硬币。

练习3.27 格雷码有一个有用的特性，即连续的数字仅在一个比特位上不同。表3.23列出了表示数字0到7的3位格雷码。设计一个3位模8格雷码计数器FSM，无输入，有三个输出。（一个模N计数器从0计数到N - 1，

表3.23 3位格雷码

Number	Gray code		
0	0	0	0
1	0	0	1
2	0	1	1
3	0	1	0
4	1	1	0
5	1	1	1
6	1	0	1
7	1	0	0

然后重复。例如，手表使用模60计数器来计数分钟和秒，从0计数到59。复位时，输出应为000。在每个时钟边沿，输出应前进到下一个格雷码。达到100后，应重复回到000。

练习3.28 通过添加一个UP输入，将练习3.27中的模8格雷码计数器扩展为可上下计数的计数器。若UP = 为1，计数器前进到下一个数字。若UP = 为0，计数器后退到上一个数字。

练习3.29 贵公司Detect-o-rama希望设计一个有限状态机，该状态机接收两个输入A和B，并产生一个输出Z。在第n个周期中，输出Z_n要么是对应输入A_n与前一个输入A_{n-1}的布尔与运算或或运算，具体取决于另一个输入B_n：

$$\begin{aligned} Z_n &= A_n A_{n-1} & \text{if } B_n &= 0 \\ Z_n &= A_n + A_{n-1} & \text{if } B_n &= 1 \end{aligned}$$

- (a) 根据图3所示的输入，画出Z的波形图。 .71.
- (b) 该FSM是Moore机还是Mealy机？
- (c) 设计FSM。展示你的状态转换图、编码状态转换表、下一状态与输出方程以及原理图。

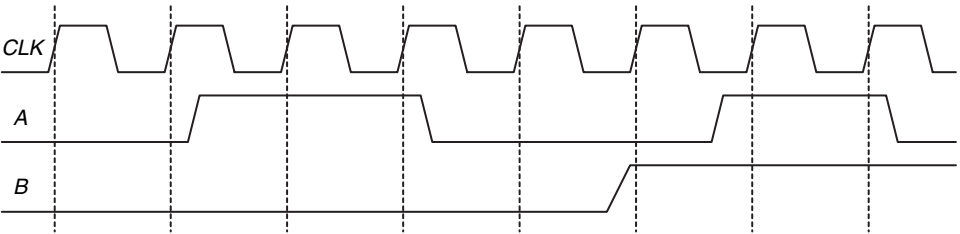
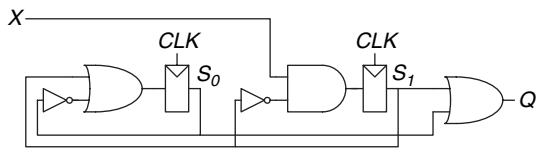


图3.71 练习3.29的FSM输入波形

练习3.30 设计一个具有一个输入A和两个输出X和Y的FSM。如果A总共（不一定是连续地）至少为1三个周期，则X应为1。如果A至少连续两个周期为1，则Y应为1。给出你的状态转换图、编码状态转换表、次态和输出方程以及原理图。

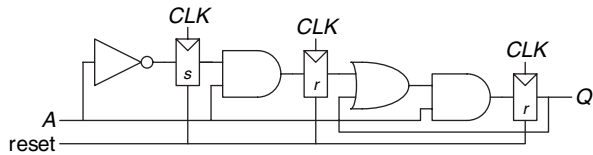
练习3.31 分析图3.72所示的有限状态机。写出状态转换表和输出表，并画出状态转换图。

图3.72 练习3.31的FSM示意图



练习3.32 对图3.73所示的FSM重复练习3.31。注意，*s*和*r*寄存器输入分别表示置位和复位。

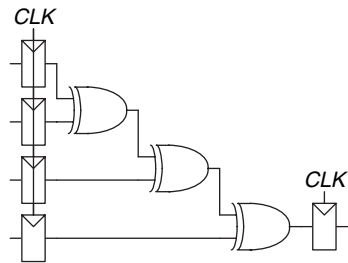
图3.73 练习3.32的FSM示意图



练习3.33 Ben Bitdiddle设计了图3.74中的电路，用于计算一个带寄存器的四输入异或函数。每个两输入异或门的传播延迟为100 ps，污染延迟为55 ps。每个触发器的建立时间为60 ps，保持时间为20 ps，时钟到Q的最大延迟为70 ps，时钟到Q的最小延迟为50 ps。

(a) 如果没有时钟偏移，电路的最大工作频率是多少？ (b) 如果电路必须在2 G Hz下工作，它能容忍多大的时钟偏移？ (c) 在可能发生保持时间违例之前，电路能容忍多大的时钟偏移？ (d) Alyssa P. Hacker指出，她可以重新设计寄存器之间的组合逻辑，使其更快*and*并容忍更大的时钟偏移。她的改进电路也使用了三个两输入异或门，但排列方式不同。她的电路是什么？如果没有时钟偏移，其最大频率是多少？在可能发生保持时间违例之前，该电路能容忍多大的时钟偏移？

图3.74 练习3.33的已注册四输入异或电路



练习3.34 你正在为速度极快的2位RePentium处理器设计一个加法器。该加法器由两个全加器构成，第一个加法器的进位输出作为第二个加法器的进位输入，如图3.75所示。你的加法器具有输入和输出寄存器，并且必须在一个时钟周期内完成加法运算。每个全加器具有以下传播延迟：从 C_{in} 到 C_{out} 或 Sum 为20皮秒（S），从 A 或 B 到 C_{out} 为25皮秒，从 A 或 B 到 S 为30皮秒。加法器的污染延迟为：从 C_{in} 到任一输出为15皮秒，从 A 或 B 到任一输出为22皮秒。每个触发器的建立时间为30皮秒，保持时间为10皮秒，时钟到 Q 的传播延迟为35皮秒，时钟到 Q 的污染延迟为21皮秒。

(a) 如果没有时钟偏移，电路的最高工作频率是多少？ (b) 如果电路必须在8 GHz下工作，它能容忍多大的时钟偏移？ (c) 在可能发生保持时间违规之前，电路能容忍多大的时钟偏移？

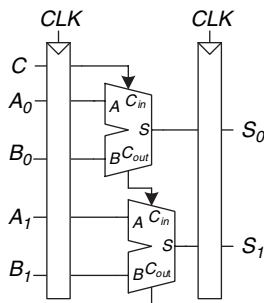


图3.75 练习3.34的2位加法器原理图

练习3.35 一种field programmable gate array (FPGA) 使用configurable logic blocks (CLBs) 而非逻辑门来实现组合逻辑。Xilinx Spartan 3 FPGA的每个CLB的传播延迟和污染延迟分别为0.61 ns和0.30 ns。它还包含传播延迟和污染延迟分别为0.72 ns和0.50 ns的触发器，以及建立时间和保持时间分别为0.53 ns和0 ns。

(a) 如果你正在构建一个需要以40 MHz频率运行的系统，在两个触发器之间最多可以使用多少个连续的CLB？假设没有时钟偏移，且CLB之间的连线没有延迟。 (b) 假设所有触发器之间的路径至少经过一个CLB。在不违反保持时间的前提下，FPGA可以容忍多大的时钟偏移？

练习3.36 一个同步器由一对触发器构成，其参数为 $t_{setup} = 50$ ps、 $T_0 = 20$ ps和 $\tau = 30$ ps。它采样一个每秒变化 10^8 次的异步输入。为使平均故障间隔时间（MTBF）达到100年，同步器的最小时钟周期是多少？

练习3.37 你需要构建一个同步器，能够接收异步输入，且MTBF为50年。系统运行在1 GHz，使用的采样触发器参数为： $\tau = 100$ ps， $T_0 = 110$ ps， $t_{\text{setup}} = 70$ ps。同步器平均每0.5秒接收一次新的异步输入（即每2秒一次）。为满足此MTBF，所需的故障概率是多少？在读取采样输入信号之前，需要等待多少个时钟周期才能达到该错误概率？

练习3.38 你正沿着走廊行走，这时遇到了迎面走来的实验室伙伴。你们俩先往同一侧让了一步，结果还是挡住了对方。接着，你们又同时往另一侧让了一步，依然互相挡着路。然后，你们俩都等了一会儿，希望对方能先让开。你可以将这种情况建模为一个亚稳态点，并应用与同步器和触发器相同的理论。假设你为自己和实验室伙伴建立了一个数学模型。你将这次不幸的相遇从亚稳态开始。在 t 秒后，你仍处于这种状态的概率为 $e^{-t/\tau}$ 。 τ 表示你的反应速率；今天，由于缺乏睡眠，你的大脑反应迟钝， $\tau = 20$ 秒。

(a) 需要多长时间才能有99%的把握确定你已经摆脱亚稳态（即弄清楚如何互相通过）？ (b) 你不仅困倦，而且极度饥饿。事实上，如果你不在3分钟内赶到自助餐厅，就会饿死。那么，

你的实验室伙伴不得不把你拖到太平间的概率？

练习3.39 你使用建立时间 $T_0 = 20$ ps和保持时间 $\tau = 30$ ps的触发器构建了一个同步器。你的上司告诉你需要将MTBF提高10倍。你需要将时钟周期增加多少？

练习3.40 Ben Bitdiddle发明了一种新型改进型同步器，如图3.76所示，他声称该同步器能在单个周期内消除亚稳态。他解释说，框M中的电路是一个模拟“亚稳态检测器”，当输入电压处于 V_{IL} 与 V_{IH} 之间的禁止区域时，该检测器会输出高电平。亚稳态检测器用于判断第一个触发器是否在D2上产生了亚稳态输出。如果是，它会异步复位该触发器，使其在D2上产生良好的0电平。随后第二个触发器对D2进行采样，始终在Q上产生有效的逻辑电平。Alyssa P. Hacker告诉Ben，该电路必然存在缺陷，因为消除亚稳态就像制造永动机一样不可能。谁是正确的？请解释，指出Ben的错误或说明Alyssa为何错误。

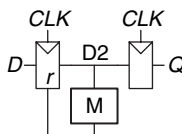


图3.76 “全新改进型” 同步器，用于练习3.40

面试问题

以下练习题展示了数字设计岗位面试中曾出现的问题。

问题3.1 绘制一个能够检测到已接收串行输入序列01010的状态机。

问题3.2 设计一个串行（逐位）二进制补码器FSM，具有两个输入 $Start$ 和 A ，以及一个输出 Q 。任意长度的二进制数从最低有效位开始提供给输入 A 。对应的输出位在同一周期出现在 Q 上。在提供最低有效位之前， $Start$ 被置位一个周期以初始化FSM。

问题3.3 锁存器和触发器之间有什么区别？在什么情况下各自更优？

问题3.4 设计一个5位计数器有限状态机。

问题3.5 设计一个边沿检测电路。当输入从 $0 \rightarrow 1$ 跳变后，输出应保持一个周期的高电平。

问题3.6 描述流水线技术的概念及其使用原因。

问题3.7 描述触发器具有负保持时间意味着什么。

问题3.8 给定信号 A ，如图3.77所示，设计一个产生信号 B 的电路。

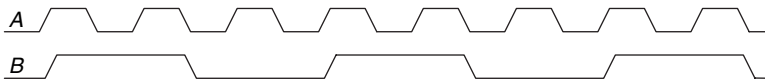


图3.77 信号波形
对于问题3.8

问题3.9 考虑两个寄存器之间的逻辑块。解释时序约束。如果在接收器（第二个触发器）的时钟输入端添加一个缓冲器，建立时间约束会变好还是变差？



HDL Dictionary	Verilog	VHDL
	$a \& b$	$a \text{ and } b$
	$a \wedge b$	$a \text{ xor } b$
	$\sim(a+b)$	$a \text{ nor } b$
Hardware		